A VISION BASED FRAMEWORK FOR REAL TIME GARMENT PIECE COUNTING AND SEWING CYCLE TIME MEASUREMENT

INDIVIDUAL THESIS

Dharmarathna P.W.A.N.

IT21255892

Vision based garment piece counting and sewing cycle time measurement

Group ID R26-IT-054

Report submitted in partial fulfilment of the requirements for the

BSc (Hons) in Information Technology

Specialization in Information Technology

Department of Information Technology

Faculty of Computing

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure

Generative AI assistance used in preparing this draft is described in Appendix E. I remain responsible for reviewing the accuracy, originality, citations, analyses and contribution claims, and for completing the disclosure required by the module before signing this declaration.

Student name: Dharmarathna P.W.A.N.
Student ID: IT21255892
Signature: _____ Date: _____

Supervisor certification

The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Supervisor: Mrs. Narmada Gamage
Signature: _____ Date: _____
Co-supervisor: Ms. Buddima Attanayake
Signature: _____ Date: _____

Abstract

Reliable piece counts and cycle times are needed to understand work at a sewing workstation. A camera can observe both machine activity and fabric handling, but repeated sewing predictions do not directly indicate how many operations have been completed. This research develops a vision component that separates workstation validation, activity recognition and completion-event reasoning. A YOLOv8n detector checks the workstation view, while an eight-frame MobileNetV3-Small classifier with temporal mean pooling distinguishes Sewing from Idle/Setup. The proposed live decoder uses smoothed probabilities, minimum state durations and a cooldown to confirm one completion. Sixty source videos contain 907 state segments and 438 manually marked events. Video-level splitting produces 6,024 clips, including 1,001 test clips from nine held-out videos. The selected classifier achieves 98.10% accuracy and 97.76% macro F1. The detector's final training-validation row reports 99.50% mAP at IoU 0.50 and 75.80% mAP across IoU 0.50 to 0.95. An unsmoothed transition baseline produces 64 candidates against 66 manual completions, with 59 matches, five false candidates and seven missed events. Event precision is 92.19%, recall is 89.39% and F1 is 90.77%; per-video count mean absolute error is 0.67 pieces. The results show useful completion information in a lightweight state sequence, while also showing that high clip accuracy does not establish reliable live counts. The individual contribution is the annotated vision experiment, state and workstation models, and event-based counting and timing design. Full-decoder cycle-time accuracy, live latency and unseen-operator generalisation require further validation.

Keywords: garment manufacturing, sewing activity recognition, garment piece counting, cycle time, workstation detection

ACKNOWLEDGEMENTS

I thank my supervisor, Mrs. Narmada Gamage, and my co-supervisor, Ms. Buddima Attanayake, for their guidance during this research. Their supervision supported the development of my individual component and the preparation of this thesis.

I acknowledge the sewing operators and workplace representatives who permitted the recordings used in the study. The research depends on observing real work, and I appreciate their cooperation. I also acknowledge my group members in R26-IT-054 for the shared project context and integration work surrounding the vision component.

The component uses published research, pre-trained models and open-source tools that are acknowledged in the references. Generative AI assistance in preparing the thesis draft is disclosed in Appendix E. The quantitative findings are based on the saved project annotations and evaluation outputs, with additional checks of their consistency during preparation.

TABLE OF CONTENTS

Declaration and Supervisor Certification i
Abstract ii
Acknowledgements iii
Table of Contents iv
List of Figures vi
List of Tables vii
List of Abbreviations viii
1 INTRODUCTION 1
1.1 Overall Project Context 1
1.2 Individual Component Context 1
1.3 Problem Statement 2
1.4 Research Gap 3
1.5 Aim and Objectives 4
1.6 Research Questions or Hypotheses 4
1.7 Scope Limitations and Assumptions 5
1.8 Individual Contribution 6
1.9 Thesis Structure 6
2 LITERATURE REVIEW AND RELATED WORK 8
2.1 Review Method and Scope 8
2.2 Domain and Theoretical Foundations 9
2.3 Related Research 10
2.4 Existing Systems Products and Techniques 11
2.5 Gap Synthesis and Conceptual Framework 12
3 RESEARCH METHODOLOGY 14
3.1 Research Design 14
3.2 Data Participants or Study Materials 14
3.3 Experimental or Study Procedure 15
3.4 Measures and Evaluation Metrics 17
3.5 Data Analysis 18
3.6 Ethics Privacy Security and Responsible AI 19
3.7 Reproducibility and Research Artefacts 20
4 PRODUCT AND SOLUTION DESIGN 22
4.1 Requirements 22
4.2 Component Architecture Within the Complete Solution 23
4.3 Design Alternatives and Decisions 25
4.4 Detailed Design 26
4.5 Standards and Professional Practice 30
5 IMPLEMENTATION INTEGRATION AND TESTING 31
5.1 Development Environment and Technology Stack 31

5.2 Component Implementation 31
5.3 Integration With the Other Components 33
5.4 Verification and Software Testing 34
5.5 Deployment and Operation 35
6 RESULTS AND VALIDATION 37
6.1 Research Results 37
6.2 Product and Technical Results 42
6.3 User or Stakeholder Validation 43
6.4 Integration Results 44
6.5 Summary of Findings 44
7 DISCUSSION 46
7.1 Interpretation of Findings 46
7.2 Comparison With Literature Baselines and Existing Solutions 47
7.3 Research Contribution 48
7.4 Product Contribution 49
7.5 Limitations and Threats to Validity 49
7.6 Implications 50
8 COMMERCIALISATION AND SUSTAINABILITY 52
8.1 Value Proposition and Target Users 52
8.2 Market and Competitive Position 52
8.3 Business or Sustainability Model 53
8.4 Scalability Intellectual Property and Compliance 54
8.5 Contribution to the Sustainable Development Goals 55
9 CONCLUSION AND FUTURE DIRECTIONS 56
9.1 Conclusion 56
9.2 Achievement of Objectives 56
9.3 Individual Research and Product Contributions 57
9.4 Future Directions 58
REFERENCES 59
GLOSSARY 61
APPENDICES 62
Appendix A Extended Evaluation Results 62
Appendix B Held Out Video Manifest 63
Appendix C Evidence Traceability 64
Appendix D Proposed Live Validation Protocol 65
Appendix E Academic Integrity and AI Use Disclosure 67

LIST OF FIGURES

Figure 4.1 Vision Component Within the Local Monitoring Architecture 24
Figure 4.2 Temporal State Classifier Architecture 27
Figure 4.3 Completion Decoder and Confirmation Rules 28
Figure 6.1 Classifier Training and Validation History 38
Figure 6.2 Confusion Matrix for Held Out State Clips 39
Figure 6.3 Detector Validation Performance Across Training 40
Figure 6.4 Manual and Predicted Completion Counts by Test Video 41

LIST OF TABLES

Table 2.1 Related Studies and Their Relevance 9
Table 3.1 Source Video and Annotation Partitions 15
Table 3.2 Generated State Classification Clips 16
Table 3.3 Model Training Settings 17
Table 4.1 Component Requirements and Verification Route 23
Table 4.2 Design Alternatives and Decisions 25
Table 4.3 Live Inference and Decoder Settings 28
Table 5.1 Information Required at the Vision Integration Boundary 33
Table 5.2 Verification Status and Remaining Tests 35
Table 6.1 Held Out State Classification Results 37
Table 6.2 Workstation Detector Validation Measurements 39
Table 6.3 Raw Completion Events by Test Video 41
Table 6.4 Measured Outcomes and Evaluation Boundaries 43
Table 8.1 Measures for a Future Commercial Pilot 54
Table 9.1 Achievement of Individual Objectives 57
Table A.1 Per Video State Classification Results 62
Table B.1 Source Videos Reserved for Testing 63
Table C.1 Research Claims and Supporting Artefacts 64
Table D.1 Live Validation Scenarios 66

LIST OF ABBREVIATIONS

AI	Artificial intelligence
BLE	Bluetooth Low Energy
CSV	Comma-separated values
F1	Harmonic mean of precision and recall
FN	False negatives
FP	False positives
I3D	Inflated three-dimensional convolutional network
IoT	Internet of Things
IoU	Intersection over union
JSON	JavaScript Object Notation
MAE	Mean absolute error
mAP	Mean average precision
RMSE	Root mean square error
TP	True positives
TSM	Temporal Shift Module
YAML	YAML Ain't Markup Language
YOLO	You Only Look Once

1 INTRODUCTION

1.1 Overall Project Context

Garment production depends on a sequence of small sewing operations. An operator may prepare a fabric piece, guide it through the machine, adjust its position and begin another sewing interval before completing the assigned operation. Production monitoring must describe this work accurately enough to support daily decisions. A piece count indicates how much work has been completed, while cycle time shows how long successive completions take. Together, these measurements can help a supervisor investigate changes in output rather than relying only on the total reported at the end of a shift.

The group research project, R26-IT-054, investigates a local system for monitoring sewing work. Its vision path observes a workstation through a fixed camera. Its operator input path records rework and downtime separately. A desktop application presents the current session and stores records for later review. The research manuscript submitted to ICAC describes this overall framework [1]. This individual thesis examines the vision component, including the evidence used to train and evaluate the models and the rules used to convert visual states into production events.

In this thesis, a garment piece means a completion of the operation being monitored at one workstation. It does not necessarily mean that an entire garment has passed through every production stage. This distinction matters when interpreting output: completing a collar operation and completing a pocket operation are different work units even if both are counted as one workstation completion. Comparisons between operators therefore require information about the assigned operation and its expected work content.

1.2 Individual Component Context

The individual component recognises two visual states, Sewing and Idle/Setup, from short video clips. It also detects whether the camera is viewing the sewing workstation. The detector and classifier serve different purposes. The detector checks the scene, while the classifier describes activity within an accepted scene. A temporal

decoder then uses the sequence of recognised states to confirm a completion event. The count increases once per confirmed event, and the event timestamps provide the basis for cycle-time measurement.

The saved research artefacts include state annotations, completion timestamps, a video split manifest, clip-generation settings, training code, model weights and evaluation outputs [2]. These artefacts allow the main classification results to be checked rather than accepted only from a summary. They also allow an unsmoothed event baseline to be reconstructed from test predictions saved as comma-separated values (CSV) files. The thesis uses this distinction throughout: a saved model result is an experimental observation, while a live processing rule is part of the solution design unless it has its own evaluation.

The final scope is camera-based recognition and production-event reasoning. Electron, FastAPI, SQLite and Bluetooth inputs are described where they affect the vision component's interfaces or the interpretation of its output. Their complete development is outside the individual contribution claimed here. The operation-type enhancement involving separate sleeves, collar and pocket inputs is also outside the evaluated experiment. The present classifier predicts operational state rather than sewing-operation category.

1.3 Problem Statement

A sewing machine can be active without a new piece being complete. An operator may pause briefly, reposition the fabric or sew several sections of the same piece. Conversely, a completion can be missed if it is identified only by a short put-aside movement. A useful monitoring method must therefore distinguish the repeated activity of sewing from a production outcome. Counting every frame labelled Sewing would produce a quantity related to viewing time rather than the number of completed pieces.

Manual observation can record completions directly, but it requires attention over the complete observation period. Machine-attached sensing provides a different form of evidence. Jung et al. used electrical-current measurements and time-series processing to estimate sewing work [3]. Such measurements can describe machine operation,

although the relationship between machine activity and an operator's full work cycle still needs interpretation. A fixed camera may capture both fabric handling and machine activity, but the images introduce problems of occlusion, visual similarity and uncertain state boundaries.

The problem addressed by this component is to infer a reviewable completion event from a continuous workstation video without modifying the sewing machine. The method must avoid repeated counts, identify an unsuitable view and maintain timestamps that can later be compared with manual observations. Its success cannot be judged only by classification accuracy. The final count and timing behaviour must also be examined because a small number of errors near transitions can change the production record.

1.4 Research Gap

Existing sewing research includes connected work measurement, visual action recognition and robotic following. VARSew provides labelled sewing actions for visual recognition [4]. Zhang et al. investigate upper-limb sewing actions to support a robot following a worker [5]. Warehouse garment counting has also been studied through laser images of hanger hooks [6]. These studies establish relevant methods, but their observation targets and outputs differ from completion monitoring at a manual sewing workstation.

The gap examined here is the connection between scene validity, lightweight state recognition and a timestamped production event. Recognising a sewing action does not by itself determine how many pieces were completed. Detecting a workstation also does not prove that the operator completed a piece. The proposed component joins these stages through explicit temporal rules so that a visual decision can be traced to the count it produces.

This is a bounded gap identified from the related work reviewed in Chapter 2. It is not a claim that no commercial product or unpublished system uses similar ideas. The contribution is the implemented combination and its evaluation on the recorded dataset. Its usefulness depends on whether the workstation's work pattern supports the selected transition rule, a question considered again in the discussion.

1.5 Aim and Objectives

The aim is to develop and evaluate a lightweight vision component that recognises sewing activity and converts supported state transitions into piece counts and cycle-time records at a fixed sewing workstation.

The specific objectives are:

O1 To prepare state-segment annotations and manual completion timestamps, and divide the source videos into separate training, validation and test sets.

O2 To train a temporal state classifier that distinguishes Sewing from Idle/Setup using short clips and a compact feature extractor.

O3 To train a single-class workstation detector and use its output to prevent unsuitable views from advancing production-event inference.

O4 To define smoothing, duration and cooldown rules for generating one completion event from a supported transition, and derive cycle time from consecutive event timestamps.

O5 To evaluate state recognition, workstation detection and a diagnostic event baseline, and identify the remaining checks needed for full live-system validation.

O6 To define outputs that can be integrated with local session monitoring while keeping vision events separate from rework and downtime inputs.

These objectives separate model development from deployment claims. O2 can be assessed through held-out clip predictions, whereas O4 requires both a clear decoder specification and a future evaluation of the complete stream. This separation makes an objective partly achieved when its design exists but its operating performance has not yet been measured.

1.6 Research Questions or Hypotheses

The component is guided by four research questions. RQ1 asks whether an eight-frame MobileNetV3-Small model can distinguish the two operational states on clips from held-out videos. RQ2 asks how much completion-event information can be recovered from the classifier's raw transitions. RQ3 asks whether workstation detection provides a suitable basis for rejecting an invalid camera view. RQ4 asks

what additional evidence is needed before the combined component can provide dependable live counts and cycle-time measurements.

These are evaluation questions rather than formal statistical hypotheses. The study contains one principal classifier experiment and one diagnostic transition analysis. It does not contain repeated independent training runs, a randomised workplace trial or a controlled comparison against every alternative model. For that reason, the thesis does not present a significance test or claim that the selected architecture is universally better than a larger video model.

RQ1 and RQ2 receive quantitative answers from the saved experiment. RQ3 receives validation-set detection evidence, with a limitation on independent testing. RQ4 is answered through the failure analysis and the live validation protocol. This arrangement connects the research questions to what can actually be established from the experiment rather than making every question depend on a single accuracy value.

1.7 Scope Limitations and Assumptions

The camera is assumed to remain fixed above the workstation and to cover the machine area, fabric handling and operator hands. The component monitors one workstation and one active session at a time. It uses a binary activity model and a single workstation class. It does not identify garment defects, assess stitch quality, recognise an operator's identity or determine whether a finished piece satisfies a customer's quality requirements.

The central process assumption is that a sufficiently stable sewing interval followed by a stable setup interval often corresponds to completion of the monitored operation. This assumption must be checked for the intended task. If an operator routinely stops and prepares the same piece between several seams, the binary sequence may resemble a new completion even though no new piece has been produced. Duration rules can reduce brief changes, but they cannot resolve every ambiguity in the work process.

The classifier dataset is divided by source video. This prevents the same source video from contributing clips to both training and testing, but it does not establish that test operators are unseen. Operator identity and operation category are not encoded in the

baseline annotations. The detector uses a different image dataset and its reported metrics come from validation during training. Live latency, full-decoder count accuracy, cycle-time error and workplace usability remain outside the completed quantitative evaluation.

1.8 Individual Contribution

My individual contribution is the vision-based monitoring component. It brings together the annotated sewing-state dataset, video-level splitting, short-clip preparation, temporal MobileNetV3-Small classification, workstation detection and transition-based production-event logic. The component is intended to supply the wider system with an operational state, view confidence, confirmed count and completion timestamps.

The research contribution also includes evaluating the outputs at more than one level. The held-out clip test establishes state recognition performance. The raw-transition replay then examines whether those states contain useful event information. Keeping these measurements separate exposes errors that would be hidden by reporting only the classifier's overall accuracy. The evidence package preserves enough intermediate information to inspect this relationship [2].

The wider application and operator controller are integration contexts. This thesis does not claim sole ownership of the complete group product, Bluetooth hardware or every backend and desktop feature. The contribution boundary is important because a successful group demonstration and a verified individual experiment answer different assessment questions. Chapter 9 maps the individual objectives to the evidence and remaining work.

1.9 Thesis Structure

Chapter 2 reviews work measurement, sewing recognition and the models relevant to the component. Chapter 3 explains data preparation, training, evaluation and research safeguards. Chapter 4 presents requirements and design decisions, including the event decoder and cycle-time definition. Chapter 5 examines implementation,

integration and testing. Chapter 6 reports the measured results and identifies the status of other validation activities.

Chapter 7 interprets the findings, compares the work with related approaches and examines threats to validity. Chapter 8 considers a possible route to workplace use, commercialisation and sustainability. Chapter 9 concludes by assessing the objectives and setting out the next experiments. The appendices provide extended results, the test-video manifest, evidence traceability, a proposed live validation protocol and an account of assistance used in preparing the thesis.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review Method and Scope

The review focuses on methods that help answer the component's design questions. The first group concerns sewing work measurement and garment counting. The second concerns visual recognition of sewing actions. The third concerns compact image and video models that could operate within a local monitoring system. Primary papers, author-published datasets and official model documentation were prioritised. Sources cited in the research manuscript were checked against their publication or dataset records where available.

Search terms included sewing work measurement, sewing action recognition, garment counting, MobileNetV3, temporal video aggregation and workstation detection. Studies were included when they described a relevant observation method, a recognition model or a measurable production output. Papers concerned only with textile defect inspection were outside the main review because the target in this component is completion timing rather than fabric quality.

This is a focused narrative review. It does not use a systematic-review protocol, and it does not report a complete database search count or a formal risk-of-bias score for every paper. Table 2.1 summarises the main domain studies and shows why their reported percentages should not be ranked directly against this experiment. The review supports design reasoning while leaving broader market and research coverage as a limitation.

Table 2.1 Related Studies and Their Relevance

Study	Observation and output	Relevance and comparison limit
Jung et al. [3]	Electrical current; sewing workload; reported 8% workload error	Supports automated work measurement; measures a different quantity
VARSew [4]	Trimmed sewing-action videos; binary and eight-class labels	Supports domain-specific action recognition; not a completion-event test
Zhang et al. [5]	Upper-limb pose for robot following; 78.7% detection accuracy and 9.1 frames per second	Shows visual sewing analysis; output and hardware differ
Martínez-Sala et al. [6]	Laser images of hanger hooks; 0.13% mean counting error	Demonstrates warehouse counting; does not measure sewing cycles
Present component [1], [2]	Workstation gate, two-state clips and raw completion transitions	Evaluates state recognition and an offline event baseline

2.2 Domain and Theoretical Foundations

Work measurement needs an operational definition of when a unit of work begins and ends. At a sewing workstation, machine motion, hand motion and garment completion are related but different observations. A supervisor may care about the interval between completed operations, while a maintenance task may care about how long the motor is active. Selecting the measurement boundary before developing the model prevents the system from returning a precise value for the wrong process.

Human activity recognition assigns labels to observed behaviour. In this component, the labels describe intervals: Sewing and Idle/Setup. A completion event is a point in time inferred from the transition between intervals. This difference explains why classification and event evaluation need separate ground truth. State segments train and test the classifier. Manually marked completion timestamps test whether the classifier sequence supports the production outcome of interest.

The model uses transfer learning, in which a feature extractor trained on a broad image task is adapted to a narrower task. MobileNetV3 was developed for an accuracy and latency trade-off on constrained devices [7]. ImageNet provides the

original image-recognition training basis [8]. The sewing experiment adapts this representation to workstation images rather than claiming that the original image classes describe sewing. The usefulness of transfer learning is assessed through the resulting held-out predictions.

Temporal evidence is obtained by aggregating frame features over a clip. Temporal Segment Networks demonstrate a general approach of sampling and aggregating observations from video [9]. The present model uses mean pooling across eight frames in a 1.5-second clip. This supplies a summary of appearance over the interval. However, averaging is insensitive to frame order, so the model should not be described as explicitly learning the direction of movement. Directional production logic is introduced later by the state decoder.

2.3 Related Research

Jung et al. developed an Internet of Things (IoT) method using current measurements, Wi-Fi transmission, Symbolic Aggregate approXimation and Dynamic Time Warping. They reported an 8% error in daily workload against actual workload [3]. The study shows how a low-cost measurement path can support smaller manufacturers. For the present problem, it also illustrates the need to define what a sensor signal means in production terms. Motor-related measurements can be useful without being equivalent to a visually confirmed operation completion.

VARSew contains 3,121 trimmed videos and 49,936 frames. Its published dataset description supports binary value-added versus non-value-added recognition and eight actions, including sewing, handling and preparing [4]. This is relevant evidence that sewing deserves a task-specific visual dataset. The baseline in this thesis uses its own recordings and annotations; it was not trained or tested on VARSew. A short action classification dataset also does not supply the same continuous completion sequence needed for a count-and-cycle evaluation.

Zhang et al. use an improved OpenPose method and upper-limb action processing for robot following. They report 78.7% detection accuracy and 9.1 frames per second [5]. Their work requires detailed motion information so that a robot can follow a worker. The component developed here requires fewer semantic states but must

connect them to a reliable event. The studies therefore illustrate different consequences of visual error: incorrect motion following in one setting and an incorrect production record in the other.

Martínez-Sala et al. count garments in warehouses by processing laser-generated images of hanger hooks, reporting a mean error of 0.13% across two warehouse tests [6]. The constrained hook geometry makes the count target physically observable. A sewing workstation does not provide the same object passage. Fabric may remain on the machine through several pauses, so the present system infers completion from activity rather than counting a rigid item crossing a sensor. The warehouse result is useful context, but its error rate is not a baseline that this video experiment has surpassed.

Taken together, these studies suggest that the observation method should be chosen around the event that needs to be recorded. Connected sensing, pose estimation and item counting each provide useful evidence under particular operating assumptions. The proposed vision component combines scene checking with a simpler operational-state model, then makes the completion assumption explicit. This allows the assumption to be tested instead of hiding it inside a single prediction label.

2.4 Existing Systems Products and Techniques

Manual tallying and stopwatch observation are practical alternatives because a human can interpret the work context. Their difficulty is sustaining attention and maintaining consistent boundaries over long periods. A manual record is still valuable as ground truth when its completion definition and timestamping method are stated. This thesis therefore uses manual completion annotations as the reference for the diagnostic baseline rather than treating automation as its own source of truth.

Simple image motion thresholds were considered as a possible starting point. They require little computation, but hands, moving fabric, camera vibration and unrelated movement can all change the image. A motion threshold cannot directly explain whether the movement corresponds to sewing or a completed operation. It may be useful for detecting scene changes, but the present implementation uses learned state recognition for the main activity decision.

Compact two-dimensional convolutional neural networks can process each sampled frame and pool features. A richer video model could learn temporal structure inside the clip. Carreira and Zisserman's inflated three-dimensional (I3D) model extends convolution into the temporal dimension [10]. Lin et al.'s Temporal Shift Module exchanges information between neighbouring frames within a two-dimensional network [11]. These are relevant alternatives for later experiments. Neither was trained as a comparison model in the recorded baseline, so no measured accuracy or speed advantage is claimed for mean pooling.

The initial development direction contained Idle/Setup, Sewing, Rest/Pause and Put-Aside. The research manuscript reports that preliminary Rest/Pause and Put-Aside predictions were not dependable enough for counting [1]. The final model therefore reduces the label set to two states. This decision trades a detailed description of actions for more stable state recognition. It also shifts responsibility to the event decoder, which must determine when a non-sewing interval represents completion.

For view validation, Ultralytics YOLOv8 provides a detection implementation with training and prediction interfaces [12]. The nano variant was selected as the workstation model. Detection localises a work area rather than identifying a completed garment. Keeping this role narrow avoids suggesting that a high detection confidence certifies the whole process. Commercial product suitability, purchase cost and factory-wide performance were not established through the literature review.

2.5 Gap Synthesis and Conceptual Framework

The conceptual framework links four forms of evidence. The camera provides a view of the work. The detector supplies a scene-validity decision. The classifier supplies state probabilities from short clips. The decoder interprets the state sequence under duration and cooldown constraints. Only the last stage creates a completion event, which can be counted and timed. Each stage has a separate failure mode and should have an appropriate evaluation.

This separation leads to a useful research design. Classification precision and recall answer whether the states are recognised. Detection metrics answer whether annotated workstations are localised. Event precision and recall answer whether

transitions align with manual completions. Count error answers whether errors accumulate over a video. Cycle-time error and live delay would answer whether the timestamps are suitable for operational decisions, but those measurements require an additional continuous-stream study.

The framework also reserves an independent path for operator-entered rework and downtime. A classifier cannot be assumed to identify whether a repeated sewing action is rework. Recording operator intent separately prevents the model's visible state from being presented as a quality judgement. The resulting system is a decision-support tool whose records should remain open to review, especially when work patterns fall outside the dataset.

3 RESEARCH METHODOLOGY

3.1 Research Design

The study uses an applied experimental development approach. A production-monitoring component is built from recorded workstation activity, and its learned parts are assessed using reserved data. Development proceeds from annotation and dataset splitting to model training, saved prediction analysis and integration design. The principal experimental unit for partitioning is the source video. The unit for classifier scoring is a clip, and the unit for completion matching is a timestamped event.

Two learned models are trained independently. The You Only Look Once version 8 nano (YOLOv8n) workstation detector learns a bounding-box task from still images. The state classifier learns a clip label from annotated video segments. Independent training makes the roles easier to inspect and avoids requiring one model to produce both a scene boundary and an operational conclusion. Their outputs are combined through deterministic rules described in Chapter 4.

The completed evaluation includes one held-out classifier test, validation results from detector training and a reconstruction of an unsmoothed event baseline. It does not include a controlled ablation of smoothing, gating or duration thresholds. The decoder settings therefore describe the intended live method rather than measured improvements over the raw baseline. This is a central boundary of the study, not an assumption that an integrated system must perform better simply because it contains more rules.

3.2 Data Participants or Study Materials

The state dataset contains 60 fixed-camera videos recorded in collar, pocket and sleeve sewing contexts [1]. The baseline annotation files contain video names and operational states, but no operator identifier or operation-type learning target. Participant counts are therefore not used to define the experimental split. The dataset supports a claim about held-out source videos from the recorded setting rather than a claim about independently held-out people.

The annotation archive contains 907 state segments and 438 manual completion events [2]. The state durations sum to 10,581.775 seconds, or approximately 2.94 hours. Sewing accounts for 7,262.651 seconds and Idle/Setup for 3,319.124 seconds. These are annotated durations, not a claim about the exact full duration of all original files. Table 3.1 summarises how these materials are distributed across the saved source-video partitions.

Table 3.1 Source Video and Annotation Partitions

Partition	Videos	Segments	Events	Annotated seconds
Training	42	635	306	7405.348
Validation	9	136	66	1586.901
Test	9	136	66	1589.526
Total	60	907	438	10581.775

The detector dataset contains 780 annotated workstation images according to the research manuscript: 546 training images, 117 validation images and 117 test images [1]. Images were auto-oriented and resized to 640 by 640 pixels. The saved YAML Ain’t Markup Language (YAML) files define one class, workstation. The archive contains model weights and detection outputs but does not contain the full image and ground-truth box sets. Consequently, the present draft can examine the training log but cannot independently recompute test-set detection metrics from predictions alone.

3.3 Experimental or Study Procedure

State annotation records each interval using video_name, start_time_sec, end_time_sec and state. The final labels are IDLE_SETUP and SEWING. Completion annotation records video_name, piece_no, event_time_sec and event_type, with NORMAL_PIECE as the recorded event type. Keeping these files separate makes it possible to test an event interpretation without replacing it with the classifier's own state labels.

The splitting script validates the annotations before assignment. It checks required columns, positive segment durations, recognised state labels and non-overlapping intervals. Event checks include an existing video reference, sequential piece numbers, chronological times and the supported event type. The script uses seed 42 and allocates source videos in a 70:15:15 ratio. It evaluates candidate assignments using state duration, event count and segment count, then refines the assignment. Balancing these quantities makes the partitions similar in workload composition, although it does not create operator independence.

Clips are generated only after videos have been assigned to partitions. Each clip lasts 1.5 seconds. The training strides are 1.0 second for Idle/Setup and 2.5 seconds for Sewing, while validation and test use a 1.5-second stride. The saved generation settings specify a boundary margin of zero, preservation of source frame rate and removal of audio. The different training strides make the two training classes close in clip count. They do not change the natural proportions in the validation and test sets.

Table 3.2 Generated State Classification Clips

Partition	Idle/Setup	Sewing	Total
Training	2023	2005	4028
Validation	302	693	995
Test	306	695	1001
Total	2631	3393	6024

Table 3.2 gives the generated clip totals. The archive reports successful generation of all 6,024 planned clips. Eight frames are sampled from each clip for model loading, and the images are resized to 224 by 224 pixels. The manuscript specifies ImageNet normalisation [1]. This preparation fixes the model's input dimensions while retaining observations from across a short interval.

The classifier is initialised from ImageNet-trained MobileNetV3-Small. Training uses AdamW, class-weighted cross-entropy, label smoothing and separate learning rates for the feature extractor and classification head. Validation macro F1 selects the checkpoint. Early stopping uses seven epochs without an improvement. The detector is fine-tuned from yolov8n.pt for 50 epochs with batch size eight and input size 640.

Table 3.3 records the principal settings so that later experiments can be compared with the same baseline.

Table 3.3 Model Training Settings

Setting	State classifier	Workstation detector
Initial model	ImageNet MobileNetV3-Small	yolov8n.pt
Input	8 frames at 224 by 224	Image at 640 by 640
Batch size	8	8
Maximum epochs	30	50
Observed epochs	12	50
Learning rates	Backbone 0.0001; head 0.0003	Ultralytics training configuration
Weight decay	0.0001	Not specified in the supplied script
Selection	Highest validation macro F1	Saved best checkpoint; validation log reported separately

3.4 Measures and Evaluation Metrics

Classifier evaluation uses accuracy, per-class precision, recall and F1, together with macro and weighted F1. Accuracy is the number of correct clip predictions divided by all evaluated clips. For a class, precision is TP divided by TP plus FP, recall is TP divided by TP plus FN, and F1 is twice precision times recall divided by their sum. TP, FP and FN denote true positives, false positives and false negatives. Macro F1 gives each class equal weight; weighted F1 gives more influence to the class with more clips.

Both forms are needed because Sewing accounts for 695 of the 1,001 test clips. A model that always predicts Sewing would have 69.43% test accuracy but would recognise no Idle/Setup clips. This is a simple derived reference, not a separately trained baseline. The confusion matrix is therefore examined alongside accuracy so that the result cannot conceal weak recognition of the less frequent state.

Detector evaluation uses precision, recall and mean average precision, abbreviated mAP. Intersection over union, or IoU, is the overlap area of a predicted box and its reference box divided by their combined area. mAP at IoU 0.50 permits looser overlap than mAP averaged over IoU thresholds from 0.50 to 0.95. A detector can score highly at the first threshold while still having difficulty with tighter localisation. The reported detector values are validation measurements, with checkpoint interpretation stated in Chapter 6.

Event evaluation uses one-to-one matching within a tolerance of plus or minus two seconds. Raw predictions are ordered within each test video. A candidate is placed at the start time of the first predicted Idle/Setup clip after a predicted Sewing clip. Each candidate is matched to the nearest still-unmatched manual event inside the tolerance. Matched candidates are TP; unmatched candidates are FP; unmatched manual events are FN. The event precision, recall and F1 use the same definitions as classification but refer to completions rather than clips.

Per-video count error is predicted candidates minus manual events. Count mean absolute error, or MAE, averages the absolute value of this difference over nine videos. Root mean square error, or RMSE, is also derived in the extended analysis to show the effect of larger count errors. Matched-event timing MAE averages absolute timestamp differences only over matched events. It excludes missed and false events and does not measure capture-to-display delay. A full cycle-time MAE would require correctly paired consecutive completions and is not inferred from this statistic.

3.5 Data Analysis

The evaluation summary, class report and confusion matrix are checked against the 1,001 rows in test_predictions.csv. Correctness is recomputed from actual_state and predicted_state. The same file is grouped by video to inspect whether errors are evenly distributed. Annotation durations and event totals are checked against the split manifest. These checks establish internal consistency between the reported results and their saved evidence.

The event baseline is reconstructed independently of the live decoder. Sorting uses the clip start timestamp. A raw Sewing-to-Idle/Setup change creates one candidate,

and matching proceeds chronologically using the two-second tolerance. The replay reproduces the manuscript's 59 matched events, five unmatched candidates and seven missed events. Per-video counts and matches are retained because an aggregate total can hide compensating errors.

Training curves are analysed in relation to the checkpoint decision. Rising training performance after the best validation epoch is treated as evidence of a possible loss of generalisation, not as a reason to replace the selected checkpoint with the final one. Detector curves are inspected separately. No inferential confidence interval is calculated by treating adjacent clips as independent samples, because clips within the same video share the setting and activity. A larger study should repeat training and use video- or operator-level resampling.

The analysis also distinguishes model behaviour from data preparation. Test clips are generated inside manually labelled state intervals. They are therefore less representative of a live rolling window that can span a state change. Missing short intervals and incomplete segment tails may remove transition evidence even when all remaining clips are classified correctly. This is examined in the results rather than assuming that each manual event must have a corresponding pair of test clips.

3.6 Ethics Privacy Security and Responsible AI

Operator and workplace permission were obtained for recording the sewing videos. Recording focused on the workstation and the actions needed for the research. Permission to collect video does not establish formal institutional ethics approval or authorise every later use. The study does not report an ethics approval identifier, and identifiable footage should remain subject to the agreed access and use conditions. Permission records should be retained with the research materials before wider sharing.

The model does not use face recognition or identity labels. Even so, hands, clothing, surroundings and work patterns can reveal information about an operator. A restricted research copy of the source videos is therefore preferable to unrestricted public distribution. Derived state and event records should contain only the session

information required for the monitoring task. Access, retention and deletion arrangements need to be agreed for a workplace pilot.

Production analytics can affect people even when the model only observes activity. A long cycle may reflect an operation change, rework, equipment trouble or incomplete visual evidence. It should not automatically be interpreted as poor effort. The interface should preserve uncertain states and operator-entered context, and supervisors should review disputed events with the operator. A limited dataset cannot justify a high-stakes employment decision based on an unreviewed score.

Research reporting must also preserve the difference between measured and proposed performance. This thesis reports test clips, detector validation and a diagnostic event replay as separate results. Interview findings, cost savings and full software test results are not available in the research records. Artificial intelligence (AI) assistance used in preparing the thesis is disclosed in Appendix E. Similarity checking is treated as a review of source use and attribution rather than proof that every technical claim is correct.

3.7 Reproducibility and Research Artefacts

The archive preserves several important stages of the experiment: segment_annotation.csv, events.csv, split_manifest.csv, generation_config.json, training_config.json, model.py, training scripts, best weights and classifier predictions. The label mapping fixes Idle/Setup as index zero and Sewing as index one. File-level settings and outputs provide a trace from annotation to the results presented in this thesis. Appendix C records the relationship between these artefacts and the claims they support.

The splitting and training scripts set random seeds. The generation configuration also stores hashes of the partition annotation files. These measures help identify the intended experiment, but they do not make every run exactly reproducible across different devices and package versions. The training code is designed to record runtime versions, while the complete environment record and all imported source modules are not included in the supplied archive. Raw videos, generated clips and detector ground-truth images are also absent from this evidence copy.

A complete reproducibility release would preserve a dependency lock file, device details, the exact source revision, dataset permissions and the missing data-loading and metric modules. Model files should be loaded from a trusted research source, and checkpoint dimensions must agree with the architecture. The available archive is sufficient to reproduce the reported classifier metrics and event replay from saved predictions, but not to rerun the full training and live application end to end without additional materials.

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

The component must produce records that a user can interpret and a researcher can test. Its primary functional requirements are to accept camera frames, detect the workstation, maintain a short frame history, estimate the operational state and create one event when a completion rule is satisfied. The output must include timestamps, because a count without its event sequence cannot support cycle-time analysis or a later investigation of duplicate and missed completions.

The scene gate must prevent a camera aimed away from the workstation from advancing the event logic. An uncertain state must remain distinguishable from Idle/Setup. Starting, stopping or changing a session must reset transient inference history so that activity from one session cannot create an event in another. Integration must identify whether an event came from vision or an operator input. These are design requirements; the extent to which each is verified is reported separately in Chapter 5.

The non-functional requirements include responsive local processing, readable model outputs, reproducible settings and controlled access to recordings. The design avoids depending on an internet connection for every inference result. However, a lightweight architecture does not by itself prove an acceptable frame rate or display delay. Performance requirements need a defined target device and a measured operating workload before they can be marked as satisfied. Table 4.1 connects the requirements to the available evidence and the next validation activity.

Table 4.1 Component Requirements and Verification Route

ID	Requirement	Verification route
F1	Recognise Sewing and Idle/Setup	Held-out clip test and class report
F2	Detect the workstation before state updates	Detector validation plus planned invalid-view stream test
F3	Confirm one event per supported completion	Raw event replay plus planned full-decoder replay
F4	Derive cycles from consecutive valid events	Timestamp logic review plus planned cycle-time comparison
F5	Keep session and event sources distinct	Integration contract review and planned persistence checks
N1	Operate responsively on the target device	Planned capture-to-display latency measurement
N2	Preserve research traceability	Saved annotations, configuration, model and prediction records

4.2 Component Architecture Within the Complete Solution

Figure 4.1 shows the vision component within the group solution. A fixed camera supplies video to a workstation detector. Accepted observations enter a 1.5-second frame buffer. The temporal classifier produces two class probabilities, and a smoother supplies a stable decision or an uncertain status. The event decoder then determines whether the sequence supports a completion. FastAPI receives the resulting state and events, maintains session context and publishes updates to the Electron interface. SQLite provides local storage [1].

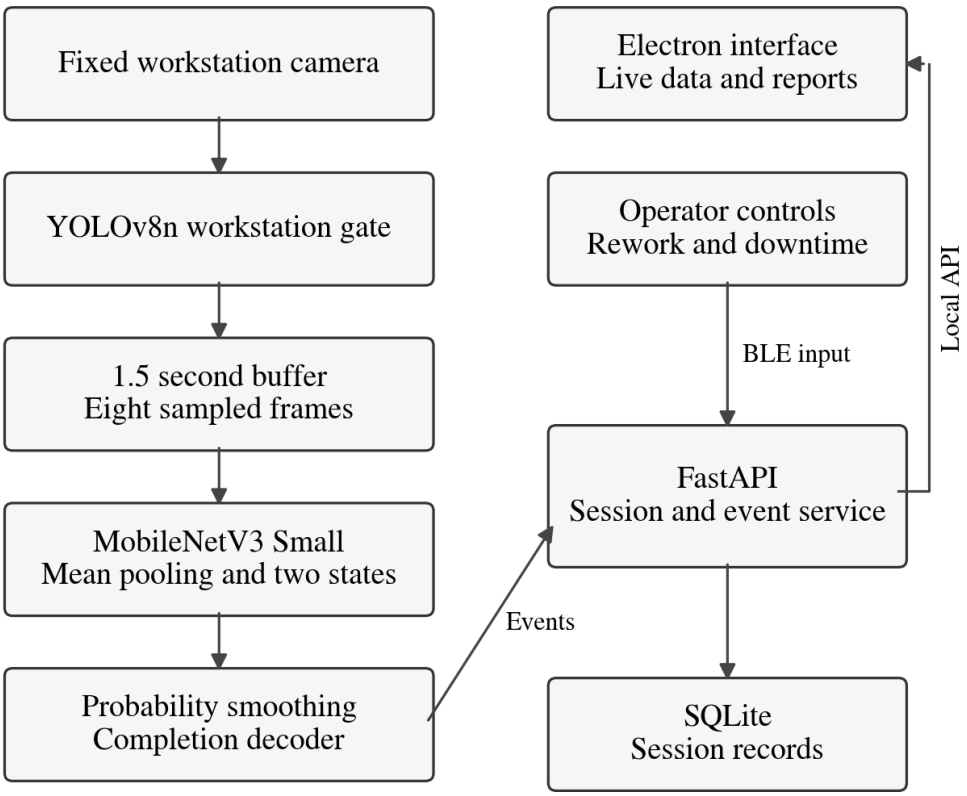


Figure 4.1 Vision Component Within the Local Monitoring Architecture

Source: project design or analysis of saved research artefacts [1], [2].

The operator controller supplies a separate input stream through Bluetooth Low Energy, abbreviated BLE. The manuscript describes an ESP32-C3 controller for rework, downtime and reset [1]. These inputs modify the interpretation or accumulation of production events; they do not change the classifier's visual probabilities. The architecture should therefore preserve both sources rather than combining them into an unexplained count correction.

The vision component's boundary ends at its state and event outputs. It is responsible for the relationship between frames, probabilities and completions. Session creation, employee management, historical reporting and controller connection management belong to the wider application. This boundary makes it possible to test inference

independently from the desktop interface and to investigate a count error without assuming that the display caused it.

Workstation detection is a gate in the reported method. The bounding box and confidence are made available to the interface, but the experiment does not establish a detector-cropped classifier training pipeline. A future change that crops all classifier inputs to the detected box should be treated as a new preprocessing choice and evaluated accordingly. Otherwise, a deployment could silently present the classifier with images different from those used during training.

4.3 Design Alternatives and Decisions

The initial label scheme attempted to describe four actions. Put-Aside was attractive because it appeared to correspond directly to completion. In practice, the manuscript reports that brief put-aside and rest predictions were unreliable [1]. Restricting the classifier to Sewing and Idle/Setup made the activity task more stable. The decoder then inferred completion from a supported transition. This was a practical response to the available data, although it reduced the semantic detail of the output.

The temporal model uses a shared two-dimensional frame extractor with mean pooling. It was selected to keep the representation compact and avoid introducing a large recurrent or three-dimensional network into the initial local pipeline. The choice is supported by its held-out results, but the study has no matched architecture comparison. Table 4.2 records alternatives and the particular trade-offs that would need testing before a replacement is justified.

Table 4.2 Design Alternatives and Decisions

Decision	Selected approach	Alternative and trade-off
State labels	Sewing and Idle/Setup	Four actions offer more detail but add brief ambiguous labels
Temporal model	Shared MobileNetV3-Small with mean pooling	Recurrent or 3D model may capture order but needs a separate evaluation
Completion logic	Explicit state transition and duration rules	Direct event model may learn completion but needs event-centred examples

Decision	Selected approach	Alternative and trade-off
Scene checking	Single-class YOLOv8n gate	Fixed image-region rules are simpler but less adaptable to view changes
Processing location	Local inference path	Remote inference introduces network dependence and footage transfer
Timing basis	Consecutive completion timestamps	Session-start timing makes the first partial cycle hard to interpret

The scene gate was chosen because a confident classifier output is not sufficient evidence that the camera is observing the right place. A classifier forced to choose between two states may still return a state for an unrelated scene. Requiring a workstation detection first gives the system a separate rejection decision. Negative views and camera obstructions still need a continuous-stream test because positive workstation detection scores do not establish rejection behaviour by themselves.

The final application uses local session storage in the architecture reported by the manuscript. This supports work without continuous cloud access and reduces the need to transmit video externally. The architecture is a suitable integration direction, but data backup, access control and recovery remain operational requirements. Local processing changes where a risk is managed; it does not remove the risk of losing or exposing records.

4.4 Detailed Design

The state model receives an input tensor with batch, time, channel, height and width dimensions. With eight sampled colour frames at 224 by 224 pixels, the input shape is B by 8 by 3 by 224 by 224. The frames are reshaped so that the same MobileNetV3-Small feature extractor processes them. Spatial average pooling produces a 576-value feature vector for each frame. Averaging over eight vectors produces one representation per clip.

Figure 4.2 shows the architecture implemented in model.py. The pooled representation passes through a linear layer with 1,024 units, Hardswish activation, dropout of 0.20 and a two-unit output layer. The forward method returns logits. Softmax converts them into probabilities during inference. The model therefore

separates the learned classifier output from the confidence threshold later used to accept a stable state.

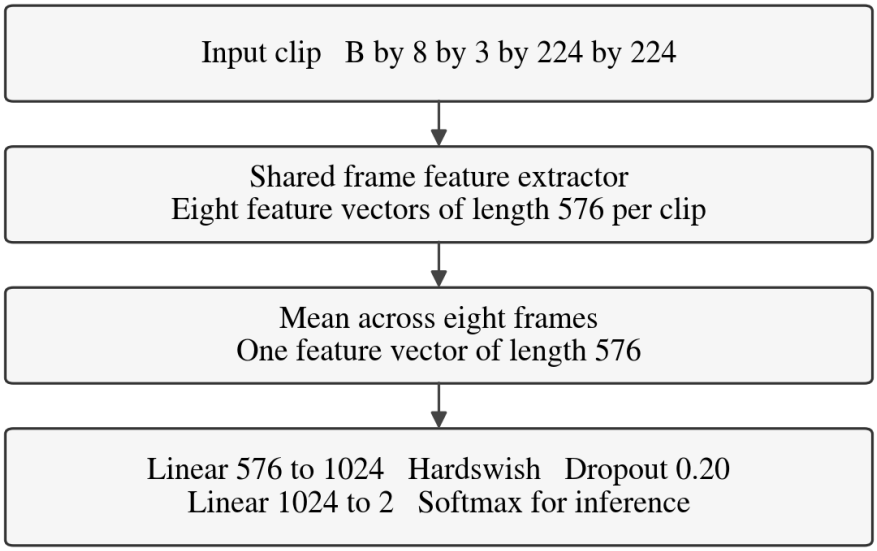


Figure 4.2 Temporal State Classifier Architecture

Source: project design or analysis of saved research artefacts [1], [2].

Mean pooling gives each sampled frame the same weight. Reversing the order of the sampled frames leaves the pooled representation unchanged. The model can recognise a pattern of sewing-related appearances over the clip, but it does not represent an explicit before-and-after ordering inside that clip. The surrounding decoder introduces sequence order by processing successive model outputs. This is why the complete method should be described as a temporal pipeline rather than an order-sensitive clip network.

The live specification evaluates the workstation confidence against 0.50. If the view is rejected, the probability history is cleared and no update is supplied to the decoder. Accepted views support inference at approximately 0.3-second intervals. The smoother averages the most recent five probability vectors. A stable state is selected only when the largest averaged probability reaches 0.55. Otherwise, the status is Uncertain [1]. These settings are recorded configuration choices rather than independently optimised thresholds.

Figure 4.3 summarises the completion rule. The decoder begins unarmed. Stable Sewing for at least 2.0 seconds arms it. A subsequent Idle/Setup interval must remain stable for at least 0.6 seconds before one event is confirmed. Confirmation increments the count, stores a timestamp and disarms the decoder. A 3.0-second cooldown restricts repeated events. Table 4.3 lists the values that define this behaviour.

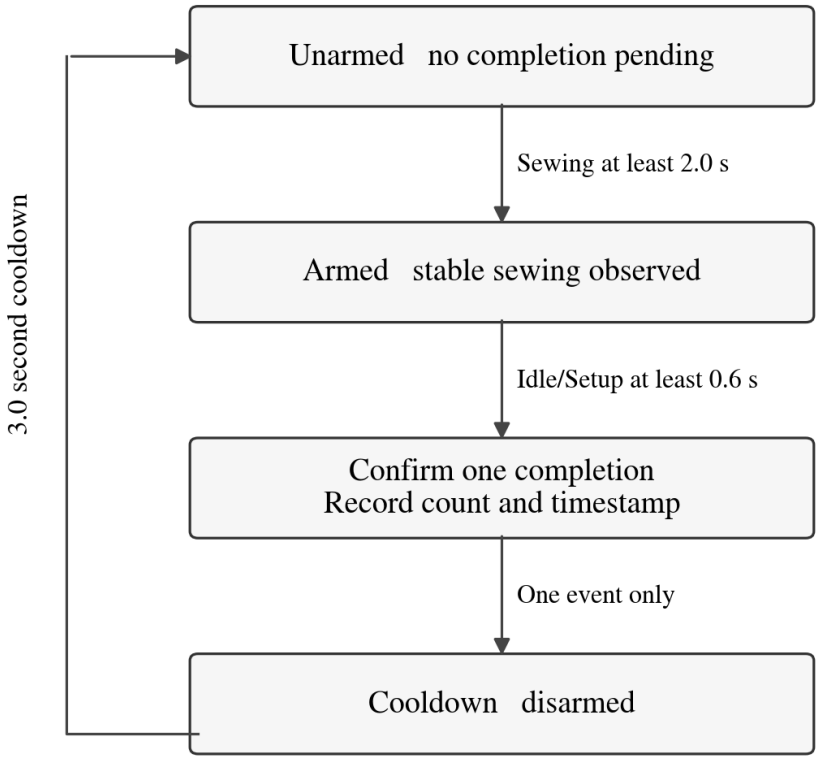


Figure 4.3 Completion Decoder and Confirmation Rules

Source: project design or analysis of saved research artefacts [1], [2].

Table 4.3 Live Inference and Decoder Settings

Setting	Value	Purpose
Clip history	1.5 seconds	Short visual context
Frames per inference	8	Fixed model input
Inference interval	Approximately 0.3 seconds	Scheduled state updates

Setting	Value	Purpose
Workstation confidence	At least 0.50	View acceptance
Smoothing history	5 probability vectors	Reduce isolated changes
Stable-state confidence	At least 0.55	Retain uncertain decisions
Stable Sewing	At least 2.0 seconds	Arm completion logic
Stable Idle/Setup	At least 0.6 seconds	Confirm the transition
Cooldown	3.0 seconds	Limit repeated events

An invalid gate must not be interpreted as Idle/Setup. The manuscript specifies clearing probability history and withholding decoder updates. This does not establish that every armed decoder timer is reset. Recovery after a long camera interruption is therefore an important test case: the system must not join unrelated activity across the interruption and emit a stale completion. A conservative recovery policy could require a new valid sewing interval, but that policy needs to be implemented and evaluated before it is described as current behaviour.

If completion timestamps are t1, t2 and later values, the second and subsequent cycle times are the differences between consecutive timestamps. The first completion has no preceding completion and therefore has no completed cycle interval. A session average should use subsequent valid intervals and state clearly how held downtime is treated. This definition follows the final manuscript [1], rather than the earlier development plan's session-start estimate for the first event.

The basic timestamp difference measures elapsed completion-to-completion time. The wider system's downtime mode is intended to pause productive-time accounting, while rework mode suppresses count accumulation until reset. These are separate interpretations of the event stream. A cycle with downtime should retain enough information to distinguish elapsed time from productive time. Without explicit hold intervals and a live comparison, the thesis cannot claim that the stored productive cycle time is already quantitatively validated.

4.5 Standards and Professional Practice

The design uses explicit model dimensions, label mapping and configuration files so that a saved checkpoint can be interpreted correctly. A classifier with a different hidden-layer size is not interchangeable with the recorded 1,024-unit head. Model loading should check the expected labels and dimensions before accepting the file. This reduces the chance that an application runs with a model whose outputs are mapped to the wrong operational states.

Time handling should distinguish elapsed-time measurement from displayed dates. A monotonic clock is suitable for durations because wall-clock adjustment can change calendar timestamps. Session records also need a consistent calendar-time convention for export and comparison. The live implementation should preserve both where necessary. This is a professional design requirement; the supplied archive does not establish that every runtime path currently uses it.

Research traceability also requires a record of settings that affect event behaviour. The workstation threshold, smoothing window, confidence threshold, minimum durations and cooldown should be associated with a model and decoder version. A later change may improve some work patterns while suppressing short valid operations. Keeping the configuration with the session makes the effect of such a change reviewable rather than leaving unexplained differences in production totals.

The prototype is a monitoring tool and should remain separate from machine safety control. The detector's confidence does not certify a safe workstation, and a missing completion does not prove that an operator failed to complete the work. Appropriate access, licensing and workplace approval must be addressed before distribution. Chapter 8 considers these requirements without treating a commercial plan as evidence that the prototype has passed an industrial acceptance test.

5 IMPLEMENTATION INTEGRATION AND TESTING

5.1 Development Environment and Technology Stack

The research implementation uses Python, PyTorch and torchvision for the state model. Video preparation and inference involve frame loading, resizing and normalisation. Ultralytics provides the YOLOv8n training path. Configuration is stored in JavaScript Object Notation (JSON) or YAML, and experiment outputs are retained as CSV and JSON files. These formats allow a reviewer to examine class counts, predictions and training history without relying on the desktop interface.

The wider architecture uses a FastAPI service, an Electron client and SQLite session storage [1]. The classifier training script can select CUDA, Apple Metal Performance Shaders or CPU according to availability. The detector training script specifies device zero, which is a requested training device rather than a complete hardware specification. No target-device speed comparison is inferred from these settings.

Exact Python, package, operating-system and processor versions are important for a final reproducibility record. The training code includes a path for recording Python, PyTorch and torchvision versions, but the associated complete environment output is not present in the supplied evidence copy. Technology choices are recorded here, while the exact hardware and package versions remain to be confirmed. A final release should preserve the successful environment and the complete project source with the models.

5.2 Component Implementation

The annotation and splitting stage protects the relationship between videos and labels. The split manifest records each source video's partition, segment count, event count and state durations. Since splitting occurs before clip generation, the many overlapping or nearby training clips do not create cross-partition source-video leakage. The balancing procedure uses metadata quantities rather than model scores, which keeps model performance from influencing the reserved test assignment.

The clip generator reads labelled intervals and creates fixed-duration clips using partition-specific strides. The generation configuration preserves the clip counts and source annotation hashes. This provides a useful check if clips are regenerated later. A change to the label boundaries, duration or stride should produce a distinct experiment record instead of being treated as the same dataset simply because the source videos have not changed.

The model implementation checks that its input has five dimensions. It reshapes frames for shared feature extraction, performs spatial and temporal averaging, then applies the new classification head. Both feature-extractor and head parameters are trainable. The optimizer places them in separate parameter groups with different learning rates, allowing the pre-trained representation to change more slowly than the new task-specific output head.

The training script seeds Python, NumPy and PyTorch, constructs reproducible data-loader generators and selects a processing device. Class weights are calculated from training clip counts. Cross-entropy uses label smoothing of 0.05, and gradients are clipped to a maximum norm of five. AdamW supplies decoupled weight decay [13]. A cosine scheduler changes learning rates over the planned 30-epoch training horizon. These details can be checked directly in the saved script [2].

Checkpoint selection uses validation macro F1. If the score ties, lower validation loss is used as a secondary condition. The script writes the current checkpoint each epoch and replaces the best checkpoint when the selection condition is met. Saving uses a temporary file followed by an atomic replacement. The training history records losses, metrics, learning rates and per-epoch elapsed seconds. Seven epochs without improvement stop the run after epoch 12, leaving epoch 5 as the selected checkpoint.

The detector script loads yolov8n.pt and fine-tunes a single workstation class for 50 epochs. It requests saved weights and plots, and the resulting log records box-related losses and validation metrics. The archive's predictions.json contains detections with bounding boxes and scores. Those predictions are useful for inspection, but without corresponding reference boxes they cannot establish a new independent detection score. The reported detector measurements remain those in the training log.

5.3 Integration With the Other Components

The vision component needs a clear output contract. A state update should carry the session reference, source time, accepted-view status, workstation confidence, predicted probabilities and stable state. A completion event should carry its own identity, timestamp and count position. The distinction prevents a repeated display update from being mistaken for a new production event. Table 5.1 describes the required information at the interface level rather than asserting an undocumented database schema.

Table 5.1 Information Required at the Vision Integration Boundary

Record	Required information	Reason
State update	Session, source time, view status, probabilities, stable state	Explain the current inference decision
Completion	Session, event identity, timestamp, count position	Prevent duplicate accumulation and support cycles
Operator input	Session, source, event type, timestamp, hold status	Preserve rework and downtime context
Cycle record	Consecutive completion references, elapsed interval, hold interpretation	Support review of timing values
Configuration	Model and decoder version, thresholds and duration settings	Reproduce the session behaviour

In the group architecture, the service combines vision events with operator inputs and stores them on a shared session clock [1]. Rework suppresses new completion accumulation, downtime pauses cycle accounting and reset resumes monitoring. The visual state may still describe Sewing while a hold is active. This is acceptable when the interface explains both facts. Altering the classifier output to hide the hold would lose the distinction between observed behaviour and an operator's work context.

Session lifecycle is another integration boundary. An active session should own the camera worker and transient event state. Ending it should stop capture, flush or finish pending writes and release the device. A new session should begin with empty

probability history and an unarmed decoder. Otherwise, activity observed before a session change could affect the next employee's record. The archive does not include a full integration test log for these behaviours, so they remain specified checks rather than reported passes.

5.4 Verification and Software Testing

Verification for this thesis starts with the evidence files. The annotations and split totals agree, the classifier report matches the saved predictions and the confusion matrix sums to 1,001 clips. The raw-transition replay reproduces the reported event totals and timing error. These are completed analytical checks. They verify the saved results without claiming that the model has been retrained or that the live application has been executed in the current review.

Model-level testing should include input dimensions, label mapping, checkpoint compatibility and the use of evaluation mode during inference. Decoder testing should use constructed state sequences with known timing. A sustained Idle/Setup sequence must not create a completion from an unarmed state. Brief state changes should not satisfy the duration requirements. Repeated idle updates after a confirmed event should not produce a second count. These tests address behaviour that is not covered by clip accuracy.

The gate and session tests are equally important. An invalid view should clear the required history and prevent new events. Reconnection should not use stale probabilities. Starting a new session should not reuse the previous event's timestamp. Persistence testing should confirm that a completion is stored once even if the interface reconnects or retries an update. Table 5.2 separates completed evidence checks from tests that still require the full application and live decoder.

Table 5.2 Verification Status and Remaining Tests

Check	Expected result	Status
Annotation totals	60 videos, 907 segments and 438 events agree	Verified from saved files
Classifier report	982 correct predictions and 19 errors	Verified from saved predictions
Raw event replay	59 TP, 5 FP and 7 FN at a 2-second tolerance	Reproduced from saved files
Checkpoint loading	Architecture and label mapping agree	Specification reviewed; runtime test pending
Full decoder sequences	Duration, uncertainty and cooldown work as intended	Proposed test; no execution result reported
Invalid-view recovery	No stale completion across a camera interruption	Proposed integration test
Persistence and session reset	One stored event and no cross-session state	Proposed integration test
Live latency and cycle error	Measured against independent reference timing	Controlled deployment study pending

Software testing and model validation should remain separate in the final submission. A correct decoder can still receive wrong state predictions. A high-scoring classifier can still be embedded in a service that duplicates events. Testing both layers gives a stronger explanation of the system than either a software pass count or a model percentage alone. Appendix D provides a practical protocol for closing these gaps.

5.5 Deployment and Operation

The reported application is designed for local workstation use. Before a session begins, the camera view needs to be positioned consistently with the training data. The operator and monitored operation should be identified through the session context. The interface should show whether the view is valid and whether the

classifier has enough confidence to report a stable state. A count should be displayed as a confirmed event total rather than as a continuously accumulating state score.

During operation, a user needs to distinguish normal production, rework and downtime. The operator controls provide that context in the group architecture [1]. At session end, the system should preserve the events and their relevant configuration so that a report can be interpreted later. An export that retains only the final count would remove the information needed to examine missed events or compare cycle intervals.

Deployment validation must use the intended computer, camera and complete application. Camera capture, workstation detection, frame preparation, classification, smoothing, decoding, storage and interface updates all contribute to delay. The approximately 0.3-second scheduled inference interval is not a measured end-to-end response time. Continuous operation should also be observed for device disconnection, memory growth and database failures. These checks are needed before the prototype is accepted for unattended workplace monitoring.

6 RESULTS AND VALIDATION

6.1 Research Results

The state classifier correctly labels 982 of 1,001 held-out clips, giving 98.10% accuracy. Macro F1 is 97.76% and weighted F1 is 98.10%. The selected checkpoint is epoch 5, where validation macro F1 is 95.67%. Table 6.1 provides the per-class results. Idle/Setup has slightly lower precision, recall and F1 than Sewing, although both states are recognised well on the prepared test clips [2].

Table 6.1 Held Out State Classification Results

Class	Precision %	Recall %	F1 %	Support
Idle/Setup	97.36	96.41	96.88	306
Sewing	98.42	98.85	98.64	695
Macro average	97.89	97.63	97.76	1001
Weighted average	98.10	98.10	98.10	1001

Figure 6.1 shows the saved training and validation history. Training performance continues to increase after the selected checkpoint, while validation macro F1 fluctuates and finishes below its epoch-5 maximum. At epoch 12, training accuracy reaches 99.35%, but validation macro F1 is 94.74%. Selecting the final epoch on the basis of training performance would therefore ignore the validation evidence. The curve supports the decision to retain the best validation checkpoint without proving that early stopping is optimal for every dataset.

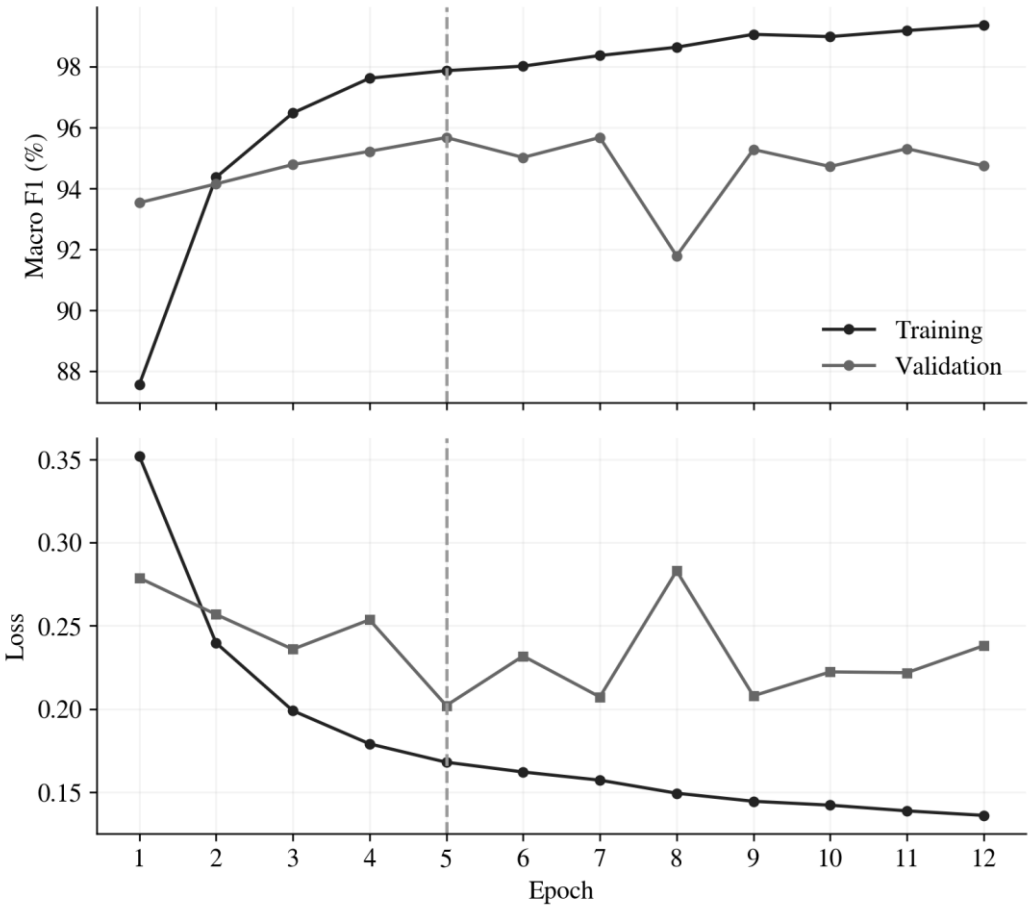


Figure 6.1 Classifier Training and Validation History

Source: project design or analysis of saved research artefacts [1], [2].

The confusion matrix in Figure 6.2 contains 295 correct Idle/Setup clips and 687 correct Sewing clips. Eleven Idle/Setup clips are predicted as Sewing, and eight Sewing clips are predicted as Idle/Setup. These 19 errors account for the difference between the measured accuracy and a perfect clip score. Their effect on production events depends on where they occur in the sequence, so the matrix alone cannot quantify counting performance.

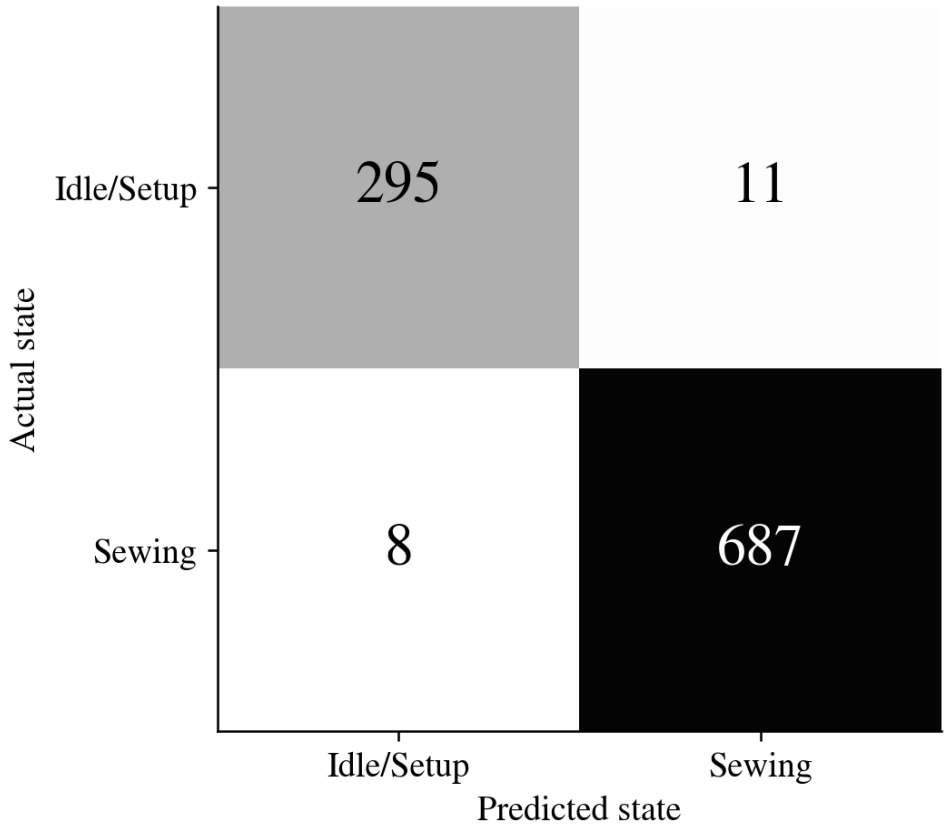


Figure 6.2 Confusion Matrix for Held Out State Clips

Source: project design or analysis of saved research artefacts [1], [2].

The detector's epoch-50 validation row reports 99.94% precision, 100.00% recall, 99.50% mAP at IoU 0.50 and 75.80% mAP averaged from IoU 0.50 to 0.95. The latter value peaks at 76.12% at epoch 39. Table 6.2 gives these readings, and Figure 6.3 shows the validation trend. The epoch-50 values should not be described as an independent test of the supplied best.pt file because the archive does not include a separate evaluation establishing that relationship.

Table 6.2 Workstation Detector Validation Measurements

Metric	Epoch 50	Interpretation
Precision	99.94%	Saved training-validation measurement
Recall	100.00%	Saved training-validation measurement
mAP at IoU 0.50	99.50%	High overlap detection at a lenient IoU threshold
mAP at IoU 0.50 to 0.95	75.80%	Stricter localisation score

Metric	Epoch 50	Interpretation
Peak mAP at IoU 0.50 to 0.95	76.12% at epoch 39	Maximum recorded value across the training log

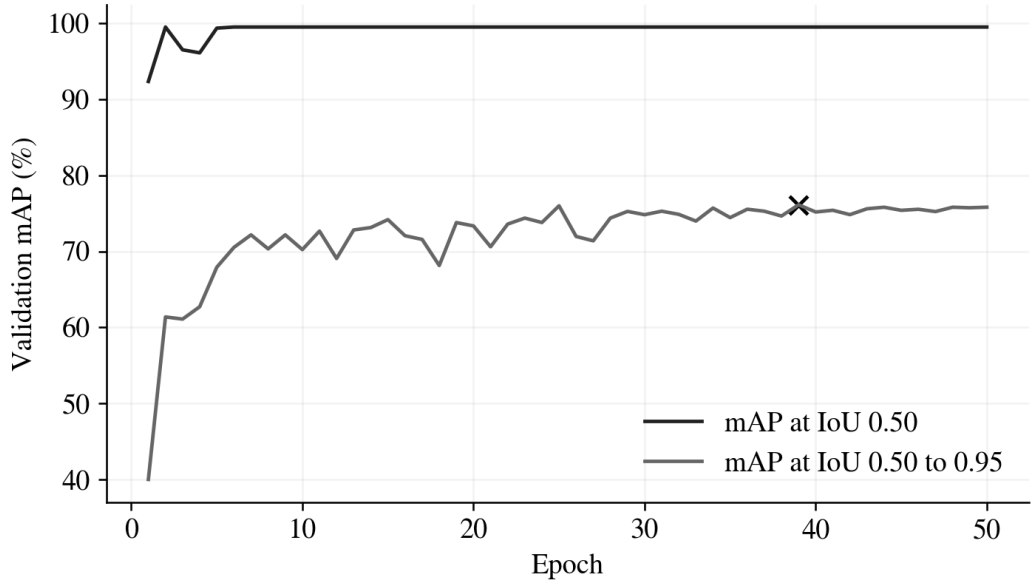


Figure 6.3 Detector Validation Performance Across Training

Source: project design or analysis of saved research artefacts [1], [2].

The raw completion baseline contains 66 manual events in nine test videos. Classifier transitions produce 64 candidates. One-to-one matching finds 59 TP, five FP and seven FN, giving 92.19% precision, 89.39% recall and 90.77% F1. Matched-event timing MAE is 0.1875 seconds, which rounds to 0.19 seconds. Per-video count MAE is 0.67 pieces. These results are reproduced from the saved predictions and event annotations, and they agree with the manuscript [1], [2].

Table 6.3 shows the event results by video. Figure 6.4 compares manual event totals with candidate totals. The mean absolute count error is based on absolute errors of 0, 1, 2, 0, 1, 0, 1, 0 and 1 across the nine videos. The derived count RMSE is 0.94 pieces, and exact totals occur in four of nine videos. These additional summaries describe the same baseline, not a second experiment.

Table 6.3 Raw Completion Events by Test Video

Video	Manual	Candidates	TP	FP	FN	Count error
v10	4	4	4	0	0	0
v12	8	7	7	0	1	-1
v18	7	9	6	3	1	+2
v22	5	5	4	1	1	0
v26	6	5	5	0	1	-1
v39	8	8	7	1	1	0
v49	12	11	11	0	1	-1
v51	4	4	4	0	0	0
v53	12	11	11	0	1	-1
Total	66	64	59	5	7	-2

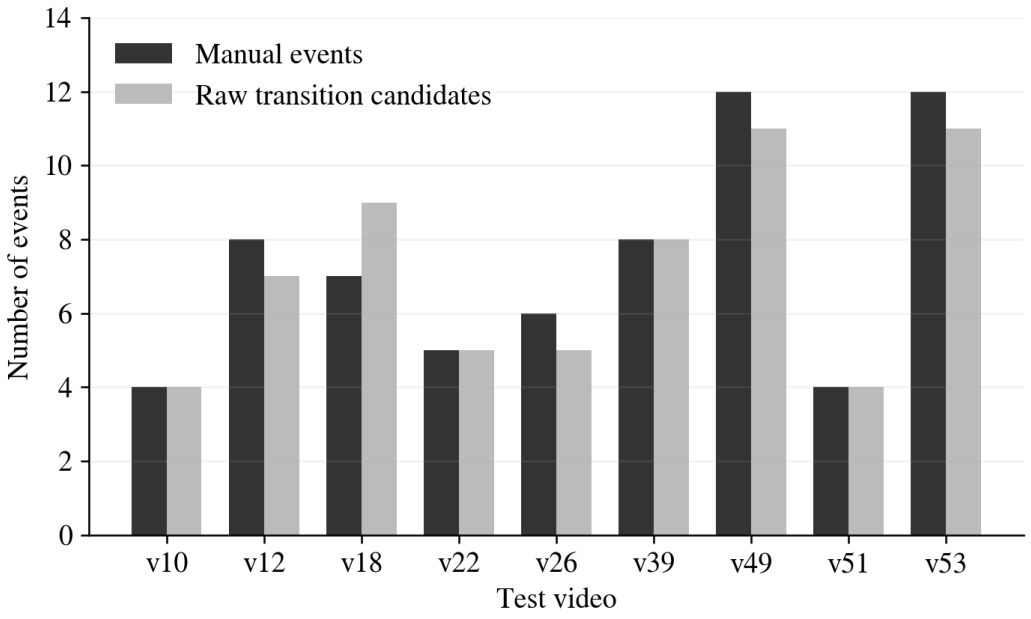


Figure 6.4 Manual and Predicted Completion Counts by Test Video

Source: project design or analysis of saved research artefacts [1], [2].

Two details show why event-level review matters. Videos v22 and v39 have correct totals, but each contains one false candidate and one missed manual event. The errors cancel numerically while leaving an incorrect event sequence. Video v26 has 100%

accuracy on its 59 test clips but still misses one completion in the raw baseline. The saved clips therefore do not guarantee complete boundary evidence, even when every evaluated clip is correctly classified.

6.2 Product and Technical Results

The main technical outcome is a saved binary temporal classifier and a saved workstation detector, together with the data and prediction records needed to inspect the experiment. The classifier's architecture, label mapping and training settings are preserved. The event specification defines how view validity, smoothed probabilities, duration rules and cooldown should operate together. The component thus provides a concrete basis for continuous production monitoring rather than only a whole-video classification result.

The experiment also establishes a useful baseline for future decoder work. Raw transitions recover most annotated completions but introduce five false candidates and miss seven events. A future full-decoder evaluation can compare against these exact videos and annotations. Such a comparison must use continuous inputs and causal timestamps. Reusing the prepared clips without recording how windows cross boundaries would not test all the conditions of live operation.

No measured target-device frame rate, memory profile, capture-to-display delay or full-decoder cycle-time error is reported. The 0.19-second matched-event error belongs to the offline raw baseline. Candidates use clip start timestamps, although a causal system must first observe enough frames to form the clip. Table 6.4 summarises the level of validation so that the available results are not mistaken for a complete product benchmark.

Table 6.4 Measured Outcomes and Evaluation Boundaries

Outcome	Evidence	Boundary
State recognition	98.10% accuracy on 1001 test clips	Prepared clips from held-out videos
Workstation localisation	Validation precision, recall and mAP	No independent detector test recomputed
Raw completion information	59 matched of 66 manual events	No smoothing, gating or duration-rule evaluation
Candidate timing	0.19-second MAE over matched events	Offline clip-start timing; no live latency claim
Cycle-time accuracy	Timestamp rule specified	No full cycle-time error measurement
Application responsiveness	Local architecture described	No target-device timing result

6.3 User or Stakeholder Validation

The recordings were collected with operator and workplace permission, but collection permission is not a usability evaluation. The research evidence does not contain a structured interview report, an acceptance questionnaire or a scored comparison of manual and automated workflows. This thesis therefore does not claim that operators prefer the interface, that supervisors saved a measured amount of time or that a factory accepted the product for routine use.

A stakeholder study should begin with the meaning of a completion at each operation. Operators and supervisors should agree on whether a sewing-to-setup transition is sufficient or whether additional evidence is needed. They should then review examples of true, false and missed events. This gives practical feedback on the consequences of error rather than asking only whether the dashboard appears useful.

The proposed validation in Appendix D combines a controlled session with a short review of the resulting records. Questions concern event trust, the handling of rework and downtime, readability and the effort required to correct a disputed count. Future findings should state participant numbers, tasks, observation duration and the method

used to collect responses. That would provide a basis for the product's workplace value without attributing opinions that have not been collected.

6.4 Integration Results

The research manuscript describes a desktop client that displays visual state, workstation confidence, piece count, cycle times and Bluetooth status, with local session records [1]. This is evidence of the reported integration approach. The model and annotation archive supports the vision experiment, but it does not include a complete application source snapshot, persisted session samples or execution logs for the integrated tests.

The integration outcome can therefore be stated at two levels. The component has defined data that can be supplied to session monitoring, and the manuscript describes how the group solution presents and stores those outputs. Quantitative performance of the combined system remains unestablished. In particular, the thesis does not treat Bluetooth reliability, rework suppression or downtime-adjusted timing as validated merely because these functions are part of the architecture.

A full integration record should link a session's events to the camera source, model configuration and operator-input sequence. Replaying the same sequence should produce the same stored completion records, including the effect of holds. This would distinguish an inference problem from a transport or persistence problem. It would also provide evidence that a displayed count is consistent with its underlying events.

6.5 Summary of Findings

RQ1 is supported by the 98.10% held-out clip accuracy and strong performance for both operational states. RQ2 is supported by 59 matched raw transitions and an event F1 of 90.77%, with identifiable missed and false completions. RQ3 is supported by strong workstation validation metrics, but requires independent-video and invalid-view testing before a broad rejection claim. RQ4 leads to a clear next step: evaluate the complete continuous pipeline, including latency, counts, cycles and hold behaviour.

The results support a lightweight state-based approach within the recorded setting. They also show that state recognition and production measurement have different failure conditions. A high clip score is useful evidence for the model, while event-level errors identify the work still needed for the product. The next chapter develops the implications of this difference for generalisation, work-process assumptions and future model decisions.

7 DISCUSSION

7.1 Interpretation of Findings

The classifier results indicate that the two operational states have distinguishable visual evidence in the recorded setting. The model correctly recognises most Idle/Setup and Sewing clips, and the macro F1 remains close to overall accuracy. The result is therefore more informative than a high score obtained by predicting only the majority class. The selected feature extractor and short-clip representation are adequate for this binary task on the prepared held-out videos.

However, production measurement depends on sequence boundaries. A false Idle/Setup prediction within a sewing interval can introduce an extra raw transition. An Idle/Setup interval predicted as Sewing can remove a valid transition. These errors need not occur often to affect the count. Video v18 illustrates the consequence: its clip accuracy is 94.90%, yet the raw sequence produces three false candidates and misses one actual completion. The count increases by two beyond the manual total, but the event record contains four matching errors.

The lower event F1 of 90.77% compared with clip macro F1 of 97.76% is consistent with this sensitivity. The two metrics measure different tasks and should not be subtracted as though they represent the same kind of error. The clip task asks whether an interval has the correct state. The event task asks whether a change is placed near a specific completion and matched only once. A monitoring system needs the second result because its records are built from events.

The matched-event timing result also needs careful interpretation. Candidates are timestamped at the start of a prepared Idle/Setup clip, and clips are generated within manually annotated intervals. This can place a candidate close to a labelled boundary without measuring when a live system could first emit it. The classifier needs enough observed frames to form a clip, and the complete decoder adds smoothing and confirmation. The 0.19-second offline error consequently supports alignment of matched candidates, but does not support a claim of a 0.19-second live response.

Cycle-time accuracy cannot be derived from this timing MAE. A missed event can merge two true cycles into one predicted interval. A false event can split a true cycle into two. Even small timestamp errors on the matched subset do not account for these changes. The correct next evaluation must compare the complete sequence of cycles and state how unmatched events are handled, rather than measuring only the convenient intervals that remain after errors are removed.

7.2 Comparison With Literature Baselines and Existing Solutions

The component follows the motivation of sewing work-measurement research by seeking more frequent and reviewable production information. Its camera-based observation differs from Jung et al.'s electrical-current method [3]. The principal trade-off is the type of evidence. Current monitoring captures machine-related activity, while the camera captures the workstation scene and visible handling. Both still require a process interpretation before their signals become an operation count.

VARSew and the pose-based sewing study show that visual recognition can be organised around more detailed actions [4], [5]. The present binary model has a narrower label set. That makes the state task easier to interpret but leaves some completion ambiguity to deterministic rules. The thesis therefore does not claim that its higher numerical clip score demonstrates a superior action-recognition system. The datasets, labels and intended outputs are different.

The warehouse laser counter observes a physical item passage [6]. Its low reported error is relevant to garment counting in a controlled transport setting, but the sewing workstation lacks an equivalent rigid counting target. Inferring an operation from state transitions is more dependent on the operator's work sequence. A direct numerical comparison would conceal this difference rather than help explain the proposed system.

MobileNetV3, temporal aggregation and YOLOv8 provide building blocks rather than a complete garment-counting solution [7], [9], [12]. The contribution lies in their task-specific preparation and event interpretation. I3D or a Temporal Shift Module would be reasonable future model comparisons [10], [11], especially if short boundaries remain difficult. The current results do not show whether either would

improve counts enough to justify a change in computational cost or implementation complexity.

7.3 Research Contribution

The first research contribution is an annotated workstation dataset with separate interval and event labels. State recognition and completion evaluation can therefore be examined without defining correctness solely through the model's own outputs. The source-video split preserves a meaningful separation between training and testing. This is stronger evidence than a random split of neighbouring clips from the same original video, although operator-level separation remains a later requirement.

The second contribution is a lightweight state model adapted to the sewing task. Shared frame extraction and mean pooling produce a fixed representation from eight frames. The held-out results show that this representation can distinguish the two states in the recorded setting. The study does not depend on a specialised sensor attached to the sewing machine, and it retains class probabilities for later temporal processing.

The third contribution is the explicit treatment of completion as an event rather than a count of state predictions. The raw replay establishes how much event information exists in the classifier sequence before the live rules are applied. Its failure analysis shows that exact totals can hide incorrect timing records and that perfect clip predictions can coexist with a missed completion. These observations help define the next experiment instead of making the thesis depend on a single headline result.

The novelty claim remains an engineering research contribution within the reviewed scope. The study combines known models with a task-specific dataset and an interpretable event path. It does not introduce a new general-purpose neural architecture or establish a universal solution to every sewing operation. Its value is the relationship it demonstrates between recognised activity and a production record that can be checked against independent annotations.

7.4 Product Contribution

The component creates the information needed for a session-based monitoring application: state, view confidence, confirmed events and completion timestamps. These outputs can support a supervisor who needs to examine when output changed rather than only the final total. Preserving uncertain states and invalid views can also help explain gaps in a record. This is a proposed practical benefit of the design, not a measured saving in supervisory time.

Keeping rework and downtime separate is another useful product decision. Visual activity alone cannot establish whether a piece is new work or a repair. The operator-input path gives the wider application additional context without forcing a binary classifier to make a quality judgement. A useful report should show how those inputs affected counts and cycle intervals so that the user can review the result.

The product contribution is incomplete until the full pipeline is tested under the intended work conditions. The component is a research prototype with encouraging offline evidence. Acceptance for routine use depends on continuous-stream performance, clear operation definitions, stable persistence and a workable correction process. This boundary protects the interpretation of the measured research without reducing the value of the implementation already completed.

7.5 Limitations and Threats to Validity

The strongest data limitation is the absence of an operator-independent split. Different source videos can still contain the same person, camera position and machine. Source-video separation prevents direct video leakage, but repeated visual context may make the held-out task easier than deployment at an unfamiliar workstation. The baseline annotation schema also lacks operation-type labels, so performance cannot be reported separately for sleeves, collar and pocket from these files.

The clip-generation method introduces another limitation. Clips lie inside labelled intervals rather than following a completely unsegmented stream. A 1.5-second clip that would cross a boundary in live use is therefore not necessarily represented in the test. Short segments and unused segment tails can leave a manual completion without

adequate clip evidence. Video v26's perfect clip score and missed event illustrate why the event test and full-stream test need different inputs.

The detector split is at image level and may place nearby views from a source video in more than one partition [1]. Its high validation scores can reflect repeated appearance as well as model ability. The saved predictions lack the matching ground-truth boxes needed for an independent re-evaluation. A future detector study should group images by source video or workstation and include deliberately invalid views, partial occlusions and changed camera positions.

The completion rule assumes a work pattern that may not hold for multi-stage operations. A stable Sewing-to-Idle/Setup transition can be a pause within one piece. A 3.0-second cooldown may prevent duplicate events, but it could also suppress a genuinely short operation. The 2.0-second sewing requirement may reject brief valid tasks. Threshold effects should be examined on validation sessions and then assessed on untouched test sessions. The current record contains no such ablation.

Only one principal classifier run is reported. Random training variation, label uncertainty and the effect of another split are not quantified. Annotation was not accompanied by an inter-rater agreement study in the supplied evidence. Manual event times also contain judgement about the moment of completion. These limitations do not invalidate the recorded result, but they narrow the confidence with which it can be transferred to new people and tasks.

Finally, the diagnostic event replay omits the full smoother, gate, timers, cooldown and operator holds. The integrated architecture has no quantitative latency or cycle-time result in the evidence. Stakeholder acceptance and commercial viability are also unmeasured. The priority is therefore a controlled live study rather than treating the present model score as a complete industrial validation.

7.6 Implications

For research, the findings show that evaluation should follow the data through to the production outcome. Reporting only clip accuracy would miss false and absent completions. Reporting only total count would miss compensating event errors. A

stronger study should retain both state-level and event-level ground truth and should record the time when each output actually becomes available.

For implementation, the model and decoder should be versioned together. A new classification checkpoint can change transition behaviour even if its clip score improves. A new duration rule can change counts without changing the model at all. Comparing sessions therefore requires the configuration, operation context and event sequence, not simply the name of the model family.

For workplace use, the system should support review rather than replace all judgement. Disputed counts, rework and interruption periods need a clear explanation. A pilot should test one agreed operation first, then expand to other work patterns after their completion rules are checked. This staged approach follows the evidence available and gives operators a practical role in deciding whether the measurements reflect their work.

8 COMMERCIALISATION AND SUSTAINABILITY

8.1 Value Proposition and Target Users

The proposed value is a local record of workstation completions and the intervals between them, obtained without modifying the sewing machine. The primary users would be sewing-line supervisors and production planners who need to review output changes during a session. Operators would also use the status display and contextual controls. For both groups, the information is useful only if the count definition matches the assigned operation and errors can be investigated.

A first pilot should target a workstation with a consistent camera position and a repeatable work sequence. This would let the team test the completion rule before adding the complexity of different layouts and operations. Selecting this context is a proposed deployment strategy, not a claim that a customer has already agreed to purchase or adopt the prototype.

Potential benefits include less repeated manual tallying and a clearer event history for cycle analysis. The present experiment does not measure financial savings, output improvement or a reduction in supervisor effort. Those benefits need a comparison with the existing workflow over a defined observation period. A commercial offer should be based on that evidence rather than translating model accuracy directly into a productivity percentage.

8.2 Market and Competitive Position

The component would compete with manual observation, machine-attached sensors and established production-monitoring systems. Its distinguishing design choices are camera-based observation, local inference and the use of an explicit completion rule. Manual observation remains flexible; sensor methods may provide dependable machine signals; larger production systems may offer line-wide integration. A camera solution must demonstrate that its installation and event-review effort are justified for the intended user.

The thesis does not present a market-size estimate, vendor price comparison or willingness-to-pay survey. No complete current product inventory has been studied. Commercial positioning is therefore provisional. The appropriate next work is to identify a few factories or training workstations, describe their current measurement process and collect requirements before proposing a fixed package or price.

The research results suggest a useful technical starting point, but buyers would need additional evidence. They would need to know which operations are supported, what happens during camera obstruction, how missed events are corrected and who is responsible for the records. These questions concern the reliability of the complete service and cannot be answered by a high classifier accuracy alone.

8.3 Business or Sustainability Model

A possible business model is an installation and support service around a local workstation application. The package could include camera placement, operation-specific setup, model configuration and periodic review of disputed events. A workstation licence or support agreement could then cover maintenance. These are proposed options; there is no revenue forecast or tested pricing model in the current research.

The cost model should include more than the camera and computer. Installation time, operator guidance, permissions, model maintenance, backup, replacement equipment and support all affect viability. An evaluation should record the effort needed to keep the view consistent and the number of manual corrections required. A low-cost device can still produce a high service cost if it needs frequent adjustment.

Table 8.1 gives measurable questions for a pilot business assessment. The table avoids estimated financial values because no quotations or operational time study have been collected. Recording these quantities would allow a later proposal to use observed costs and benefits. It would also make it possible to decide that a particular operation is better served by another observation method.

Table 8.1 Measures for a Future Commercial Pilot

Measure	How it should be assessed	Current status
Installation effort	Time to position and configure one workstation	Not measured
Record reliability	Event precision, recall and correction frequency	Raw baseline available; live result pending
Supervisor effort	Comparable manual and assisted observation sessions	Not measured
Operating cost	Equipment, licences, energy and support records	No verified costing
Operator acceptance	Structured feedback on visibility and fairness	Study pending
Maintenance burden	Camera movement, disconnects and model-update work	Long-duration pilot pending

Long-term sustainability would require a maintained source base, a reproducible environment and a clear data-retention policy. A prototype that depends on an undocumented environment is difficult to support after graduation. The archive is a useful start, but missing application modules and runtime records should be preserved before the component becomes a service commitment.

8.4 Scalability Intellectual Property and Compliance

Adding more workstations is not only a matter of running more camera feeds. Each stream needs capture, detector inference, clip preparation and state classification. If several streams share one computer, contention may increase output delay. If each uses a separate device, installation and maintenance costs rise. A scale test should measure sustained throughput and latency at the planned number of streams rather than assuming that one-workstation results transfer directly.

The project's individual code and annotations need clear ownership and permitted-use terms. Pre-trained networks, training frameworks and third-party datasets also carry licences. Ultralytics publishes AGPL-3.0 and Enterprise licensing options [14]. Before distributing a commercial version, the team should review the applicable

terms for its code, models and deployment arrangement. This is a release requirement, not a legal conclusion about a particular business model.

The detector YAML contains a commented reference to a Roboflow dataset and a stated licence. A comment alone is not sufficient to settle the rights to all source imagery or operator footage. Recording permission, dataset sharing permission and commercial-use permission should be checked separately. Workplace approval should also state how derived records may be used, retained and corrected.

A scalable product must preserve the context behind its measurements. Different operations have different completion patterns and expected cycles. Operation information can be added as session metadata or studied as a model input in a later experiment, but it should not be introduced into the evaluated baseline without new evidence. The current contribution provides a starting point for that work while retaining the limits of the binary state model.

8.5 Contribution to the Sustainable Development Goals

The proposed system is most closely related to Sustainable Development Goals 8 and 9, concerning productive employment and industrial innovation [15]. Better visibility of workstation processes could help investigate delays and improve measurement practice. This potential contribution depends on using the records fairly and preserving operator context. A system that incorrectly penalises people would conflict with the intended value of responsible work measurement.

Goal 12 is relevant to possible reductions in avoidable rework or inefficient use of resources [15]. The present study does not measure material waste, electricity consumption or emissions. It therefore claims no demonstrated environmental saving. Future workplace evaluation could record whether the information leads to an operational change and whether that change has a measurable resource effect.

Local inference can avoid some continuous video transfer, but neither it nor a compact model establishes a lower energy footprint. Future assessment should measure device energy, hardware lifetime and maintenance together with production benefits. These goals remain directions for evaluation.

9 CONCLUSION AND FUTURE DIRECTIONS

9.1 Conclusion

This research developed a vision component for garment-operation counting and sewing cycle-time measurement at a fixed workstation. The component separates workstation detection, short-clip state recognition and completion-event reasoning. The final activity model uses two states, Sewing and Idle/Setup, after the earlier four-action formulation proved unsuitable for dependable counting in preliminary development. The proposed decoder applies confidence, duration and cooldown rules before confirming a piece.

The eight-frame MobileNetV3-Small classifier achieved 98.10% accuracy and 97.76% macro F1 on 1,001 clips from nine held-out source videos. The detector's saved validation log shows high workstation precision and recall, with 75.80% mAP across IoU 0.50 to 0.95 at the final training epoch. These results support state recognition and view localisation within the recorded setting, while leaving operator-independent and independent-video detector testing open.

The unsmoothed event baseline matched 59 of 66 manual completions, with five false candidates and seven missed events. Event precision was 92.19%, recall was 89.39% and F1 was 90.77%. Count MAE was 0.67 pieces per video. This confirms useful event information in the predicted state sequence, but it also shows why clip performance cannot stand in for production accuracy. Live latency and full-system cycle-time accuracy remain to be measured.

9.2 Achievement of Objectives

Table 9.1 connects the objectives to the evidence. Dataset preparation and state-model evaluation are completed for the recorded baseline. Workstation training is completed, while broader gate validation remains partial. The decoder and cycle definition are specified, but the combined live accuracy has not been established. This assessment states what the research has achieved and what still needs observation before an industrial claim is made.

Table 9.1 Achievement of Individual Objectives

Objective	Evidence and outcome	Assessment
O1 Data and source-video split	60 videos, 907 segments, 438 events and a 42/9/9 split	Achieved for the baseline
O2 Temporal state recognition	Saved model and 98.10% test accuracy	Achieved within the recorded setting
O3 Workstation detection and gate	Trained detector and validation log; invalid-view study pending	Partly validated
O4 Completion rules and cycles	Decoder settings and timestamp definition; full stream not scored	Designed; live validation pending
O5 Evaluate and identify limits	Classifier test, detector validation and raw event replay	Achieved at the stated evaluation levels
O6 Integration outputs	Local architecture and state/event boundary defined	Specified; complete integration tests pending

The remaining work is not simply an attempt to increase the classifier percentage. It concerns whether the same method produces dependable records under camera interruptions, ambiguous work sequences and operator holds. Completing that evaluation would directly address the gap between a successful model experiment and a usable monitoring product.

9.3 Individual Research and Product Contributions

My individual contribution comprises the camera-based state and event path. The work provides the prepared annotations and split, the binary temporal classifier, the workstation detector and the logic for converting supported state transitions into counts and timing records. The component outputs are intended for the group application's session monitoring and reporting functions.

The research contributes an evidence-based view of the relationship between classifier and event performance. Reconstructing the raw sequence shows both its usefulness and its failure conditions. The product contribution is a defined monitoring component with interpretable outputs and a place for operator context.

Neither contribution requires claiming that the entire group solution, commercial operation or full live accuracy has already been independently validated.

9.4 Future Directions

The first priority is a causal continuous-video evaluation of the complete pipeline. The detector, rolling buffer, classifier, smoother, decoder and hold logic should run together on untouched videos and live sessions. Actual emission times should be recorded alongside event timestamps. This will allow count error, event precision and recall, cycle-time error and capture-to-display latency to be assessed without relying on prepared clip boundaries.

The next priority is data coverage. Source videos should record anonymised operator and operation identifiers so that leave-one-operator-out and leave-one-operation-out tests can be designed. Detector images should be split by source video or workstation. The study should include negative views, camera movement, occlusion, short valid operations and multi-stage sewing on one piece. Annotation quality should be assessed through a second annotator on a representative subset.

Model and decoder improvements should follow the failure cases. An order-sensitive temporal model may help with boundaries, while a completion-specific model may help distinguish setup within a piece from preparation for a new one. Smoothing, confidence and duration settings should be compared using validation data before a fixed configuration is tested. A later operation-type experiment should preserve its own data and results so that its contribution can be assessed separately from this baseline.

Finally, a small workplace pilot should examine correction effort, operator understanding, supervisor use and maintenance. Permission and data-use conditions should be documented, and software tests should cover session resets, persistence and interruptions. These steps would make the next submission stronger by adding the evidence most closely connected to the proposed production use.

REFERENCES

[1] “A Vision-Based Framework for Real-Time Garment Piece Counting and Sewing Cycle-Time Measurement,” research manuscript submitted to ICAC, project R26-IT-054, 2026. Unpublished manuscript.

[2] “Research Evidance,” research artefact archive for project R26-IT-054, 2026. Includes state and event annotations, model code, checkpoints, configurations and evaluation outputs. Classifier evaluation record dated July 23, 2026.

[3] W.-K. Jung, H. Kim, Y.-C. Park, J.-W. Lee and S.-H. Ahn, “Smart sewing work measurement system using IoT-based power monitoring device and approximation algorithm,” International Journal of Production Research, vol. 58, no. 20, pp. 6202–6216, 2020, doi: 10.1080/00207543.2019.1671629.

[4] A. Fathy et al., “VARSew: A Visual Action Recognition Dataset on Garment Sewing,” Mendeley Data, version 1, Oct. 18, 2023, doi: 10.17632/37b9hg5mg7.1. [Online]. Available: https://data.mendeley.com/datasets/37b9hg5mg7/1. Accessed: Oct. 5, 2026.

[5] L. Zhang et al., “A vision-based robotic system following the human upper-limb sewing action,” Mechanical Sciences, vol. 14, pp. 347–359, 2023, doi: 10.5194/ms-14-347-2023.

[6] A. S. Martínez-Sala, J. C. Sánchez-Aartnoutse and E. Egea-López, “Garment counting in a textile warehouse by means of a laser imaging system,” Sensors, vol. 13, no. 5, pp. 5630–5648, 2013, doi: 10.3390/s130505630.

[7] A. Howard et al., “Searching for MobileNetV3,” in Proceedings of the IEEE/CVF International Conference on Computer Vision, 2019, pp. 1314–1324, doi: 10.1109/ICCV.2019.00140.

[8] O. Russakovsky et al., “ImageNet large scale visual recognition challenge,” International Journal of Computer Vision, vol. 115, pp. 211–252, 2015, doi: 10.1007/s11263-015-0816-y.

[9] L. Wang et al., “Temporal segment networks: Towards good practices for deep action recognition,” in Computer Vision ECCV 2016, pp. 20–36, 2016, doi: 10.1007/978-3-319-46484-8_2.

[10] J. Carreira and A. Zisserman, “Quo vadis, action recognition? A new model and the Kinetics dataset,” in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2017, pp. 6299–6308, doi: 10.1109/CVPR.2017.502.

[11] J. Lin, C. Gan and S. Han, “TSM: Temporal shift module for efficient video understanding,” in Proceedings of the IEEE/CVF International Conference on Computer Vision, 2019, pp. 7083–7093. [Online]. Available: https://arxiv.org/abs/1811.08383.

[12] Ultralytics, “YOLOv8,” model documentation. [Online]. Available: https://docs.ultralytics.com/models/yolov8/. Accessed: Oct. 5, 2026.

[13] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in Proceedings of the International Conference on Learning Representations, 2019. [Online]. Available: https://arxiv.org/abs/1711.05101.

[14] Ultralytics, “Ultralytics license.” [Online]. Available: https://www.ultralytics.com/license. Accessed: Oct. 5, 2026.

[15] United Nations, “The 17 goals,” Department of Economic and Social Affairs. [Online]. Available: https://sdgs.un.org/goals. Accessed: Oct. 5, 2026.

GLOSSARY

Completion event is a timestamped indication that the monitored sewing operation has been completed under the selected event rule.

Cooldown is a period following an emitted event during which another event is restricted to reduce repeated counting.

Cycle time is the interval between consecutive valid completion timestamps. Elapsed and downtime-adjusted interpretations must be distinguished.

Held-out video is a source recording reserved from model training and used for evaluation.

Idle/Setup is the final binary label for non-sewing preparation or idle activity. It is not a separate judgement about downtime or operator effort.

Mean pooling is the averaging of frame feature vectors to produce one clip representation. It does not preserve their order.

Source-video leakage occurs when clips from the same original recording appear in different experimental partitions.

Stable state is a state accepted after probability smoothing and confidence checks, rather than a single unrestricted classifier output.

Workstation gate is a scene-validity check that prevents a rejected view from advancing the production-event logic.

APPENDICES

Appendix A Extended Evaluation Results

Table A.1 reports the per-video classifier results derived from test_predictions.csv. The nine videos are the same recordings used in the event baseline. Accuracy alone does not determine whether all manual completions are recoverable, as discussed in Chapter 7.

Table A.1 Per Video State Classification Results

Video	Test clips	Correct	Errors	Accuracy %
v10.mp4	107	107	0	100.00
v12.mp4	79	78	1	98.73
v18.mp4	157	149	8	94.90
v22.mp4	125	123	2	98.40
v26.mp4	59	59	0	100.00
v39.mp4	170	167	3	98.24
v49.mp4	102	101	1	99.02
v51.mp4	93	91	2	97.85
v53.mp4	109	107	2	98.17
Total	1001	982	19	98.10

The raw event replay uses the following procedure. First, order clips by start time within each video. Second, locate every predicted Sewing-to-Idle/Setup transition and retain the new clip's start time. Third, match each candidate to the nearest still-unmatched manual event within two seconds. Fourth, count matched candidates, unmatched candidates and unmatched manual events. A manual event can be used at most once. This procedure reproduces the manuscript's results; it does not apply the live rules in Table 4.3.

The resulting totals are 59 TP, five FP and seven FN. Precision is 59 divided by 64, or 92.1875%. Recall is 59 divided by 66, or 89.3939%. F1 is 118 divided by 130, or

90.7692%. Matched-event timing MAE is 0.187508 seconds. Count MAE is six absolute errors divided by nine videos, or 0.666667 pieces. Count RMSE is the square root of eight squared errors divided by nine videos, or 0.942809 pieces.

The aggregate candidate count is two below the manual total. That net difference is not the full error: five false candidates and seven missed events are present. Four video totals are exact, but only v10 and v51 have no event matching errors. The matched-event timing value should therefore always be presented with event recall and the unmatched counts.

Appendix B Held Out Video Manifest

Table B.1 records the test split from the source-video manifest. Video identifiers are used instead of personal names. The duration column is the sum of annotated state intervals and may differ from full file duration. The partitions were assigned before clip generation.

Table B.1 Source Videos Reserved for Testing

Video	Segments	Manual events	Idle/Setup seconds	Sewing seconds
v10.mp4	10	4	57.000	109.000
v12.mp4	16	8	40.328	87.198
v18.mp4	14	7	60.576	185.837
v22.mp4	11	5	41.000	152.000
v26.mp4	12	6	32.791	65.480
v39.mp4	16	8	67.518	198.053
v49.mp4	23	12	69.980	98.367
v51.mp4	10	4	47.000	98.000
v53.mp4	24	12	82.427	96.971
Total	136	66	498.620	1090.906

The source archive's state annotations identify videos from v01.mp4 through v60.mp4. Video identity defines the split boundary; operator and operation identity

do not. A later study should add those fields and preserve the current manifest as the baseline rather than silently changing the meaning of the original test set.

Appendix C Evidence Traceability

Table C.1 links key claims to saved artefacts. Paths are shortened to the relevant filenames for readability. The classifier and detector folders contain separate files with similar purposes and should remain distinct when the research package is archived.

Table C.1 Research Claims and Supporting Artefacts

Claim or design item	Supporting artefact	What it establishes
State intervals and duration	segment_annotation.csv	Labels, timestamps and 907 intervals
Completion ground truth	events.csv	438 manually marked NORMAL_PIECE events
Source-video partitions	split_manifest.csv; create_dataset_splits.py	Membership, counts and split procedure
Clip preparation	generation_config.json; clip_generator.py	Duration, strides and generated totals
Classifier architecture	model.py; label_mapping.json	Mean pooling, 1024-unit head and label order
Training configuration	training_config.json; train_mobilenetv3.py	Optimizer settings and checkpoint procedure
Classifier test result	evaluation_summary.json; classification_report.csv; confusion_matrix.csv	Reported aggregate and class results
Independent score reconstruction	test_predictions.csv	Per-clip states, probabilities and timestamps
Training curve	training_history.csv	Twelve epochs and selection context
Detector setup and outputs	data.yaml; train_yolov8.py; results.csv; best.pt	One class, training settings and validation log
Detector predictions	predictions.json	Predicted boxes and scores without reference boxes

Claim or design item	Supporting artefact	What it establishes
Live architecture and decoder	Research manuscript [1]	Reported integration and inference specification

The complete application source, full dependency lock, original video files, generated clips, detector images and reference boxes are not part of this evidence copy. Their absence limits independent training and deployment reproduction. The saved predictions remain sufficient to audit classifier scoring and the raw event reconstruction described in Appendix A.

Appendix D Proposed Live Validation Protocol

This protocol is proposed future work. No pass result is implied. Its purpose is to evaluate the complete component under causal streaming conditions and compare the generated production record with an independent reference.

Before collection, agree on the monitored operation and the physical completion boundary with the operator and supervisor. Record the model and decoder versions, thresholds, computer specifications, camera resolution and frame rate. Confirm recording permission and decide who can view the footage. Start from an empty session with no pending decoder history. Use an independent annotator or timestamped reference video to mark completions.

Run the complete pipeline continuously rather than generating state-contained clips for testing. Preserve captured-frame time, inference completion time, decoder emission time and interface-update time where possible. Record every accepted and rejected view and every operator hold. Do not adjust thresholds on the reserved test sessions after inspecting their results. Use separate validation sessions for tuning.

The trial should include normal work, short sewing intervals, repeated adjustments on one piece, rework, downtime, camera obstruction, view movement and connection loss. Table D.1 defines the expected behaviour to inspect. Numerical acceptance thresholds should be agreed before the trial based on the operation and intended use. The clip score alone cannot define these thresholds.

Table D.1 Live Validation Scenarios

Scenario	Behaviour to inspect	Evidence to retain
Normal completion	One event for each agreed completed operation	Reference and emitted timestamps
Repeated idle predictions	No repeated event after one completion	State sequence, count and cooldown
Adjustment within one piece	No premature completion	Work video and event reasoning
Short valid operation	Duration rules do not silently remove supported work	Operation timing and missed-event record
Invalid workstation view	No production advance from an unrelated scene	Gate scores and decoder history
View restored	No stale event across interruption	Pre-interruption and recovery state
Rework hold and reset	Count suppression and resumption match intent	Operator inputs and vision events
Downtime hold and reset	Elapsed and productive intervals remain distinguishable	Hold durations and cycle records
Session end and restart	No carried state or duplicate persistence	Session identities and stored events
Long continuous session	Stable processing and readable failure status	Latency, memory and error logs

Report event precision, recall and F1 with one-to-one matching, plus count MAE by session. For cycle time, first pair completion events and preserve which intervals contain missed or false events. Report timing error on correctly paired adjacent cycles and also the number of cycles that cannot be compared because of event errors. Report median and high-percentile emission and display delay. These quantities describe both typical behaviour and occasional slow responses.

After the controlled run, ask operators and supervisors to review a small selection of correct and incorrect records. Record the participants, tasks and response method. Questions should cover the meaning of the count, how a correction is requested, the treatment of interruptions and whether the display provides enough context. Keep

usability findings separate from model scores and retain negative feedback as well as favourable responses.

Appendix E Academic Integrity and AI Use Disclosure

Artificial intelligence (AI) tools, including ChatGPT, were used to support the development of my vision-based garment counting and cycle-time measurement component. AI assistance helped with understanding technical concepts, planning implementation steps, writing and debugging code, and considering solutions to problems encountered during development.

During dataset preparation, AI assisted with developing Python scripts to process the recorded sewing videos and organise the annotations. This included preparing short video clips, checking annotation formats, and separating source videos into training, validation and test sets. The recorded videos and manual annotations provided the research data; AI assistance supported the code used to prepare those materials for model training.

During model development, AI helped me compare suitable approaches and understand the use of MobileNetV3-Small for recognising Sewing and Idle/Setup states. It also supported the implementation of the training and evaluation code and the interpretation of accuracy, precision, recall, F1 scores and confusion matrices. Suggestions for improving performance included handling class imbalance, adjusting training settings and reducing unstable predictions through probability smoothing and duration rules. AI also assisted with the YOLOv8 workstation detection pipeline used to check whether the camera was observing the intended work area.

During system integration, AI supported the development and debugging of the Python FastAPI backend and the Electron, React and TypeScript frontend used to demonstrate the vision component. Assistance covered model loading, camera processing, API communication, session handling, SQLite storage and the display of states, counts and cycle times. These activities supported the integration of my component within the wider group project.